# UTS

# PEMBELAJARAN MESIN




NAMA : M. Fasha Bintang Pratama

NIM : 202531081

KELAS : H

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom.

NO.PC : 14

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benedict Samuel Andy

## INSTITUT TEKNOLOGI PLN

## TEKNIK INFORMATIKA

## 2026/2027

# Daftar Isi

*BAB I* ..... *4*

*PENDAHULUAN* ..... *4*

**1.1 Rumusan Masalah** ..... **4**

**1.2 Tujuan Masalah** ..... **4**

**1.3 Manfaat Masalah** ..... **4**

*BAB II* ..... *5*

*LANDASAN TEORI* ..... *5*

**2.1 Klasifikasi** ..... **5**

**2.2 EDA dan Preprocessing** ..... **5**

**2.3 Decision Tree** ..... **6**

**2.4 Naive Bayes** ..... **7**

**2.5 Evaluasi Model** ..... **8**

*BAB III* ..... *9*

*HASIL* ..... *9*

**3.1 Import library** ..... **9**

**3.2 Membaca dataset** ..... **9**

**3.3 Melihat informasi dataset (EDA)** ..... **10**

**3.4 Cek data kosong, duplikat, dan nilai 0 yang janggal** ..... **11**

**3.5 Melihat sebaran kelas HeartDisease** ..... **13**

**3.6 Memisahkan fitur numerik dan kategorikal** ..... **14**

**3.7 Grafik fitur kategorikal terhadap HeartDisease** ..... **15**

**3.8 Boxplot fitur numerik terhadap HeartDisease** ..... **17**

**3.9 Preprocessing: mengganti nilai 0 dengan median** ..... **18**

**3.10 Preprocessing: mengubah data huruf menjadi angka** ..... **19**

**3.11 Heatmap korelasi** ..... **20**

**3.12 Memisahkan fitur (x) dan target (y)** ..... **21**

**3.13 Membagi data latih dan data uji** ..... **22**

**3.14 Decision Tree - model pertama** ..... **23**

**3.15 Decision Tree - mencoba beberapa max_depth** ..... **24**

**3.16 Decision Tree - model akhir dan akurasi** ..... **26**

**3.17 Decision Tree - gambar pohon dan fitur penting..... 27**

**3.18 Naive Bayes - membuat model dan prediksi..... 29**

**3.19 Naive Bayes - evaluasi dengan confusion matrix ..... 30**

**3.20 Decision Tree - confusion matrix..... 31**

**3.21 Mencoba memprediksi satu pasien ..... 32**

**3.22 Perbandingan Decision Tree dan Naive Bayes ..... 33**

***BAB IV..... 35***

***PENUTUP ..... 35***

**4.1 Kesimpulan..... 35**

***DAFTAR PUSTAKA ..... 36***

# BAB I

# PENDAHULUAN

## 1.1 Rumusan Masalah

1. Bagaimana mengolah dan memahami dataset Heart Disease melalui EDA dan preprocessing
2. Bagaimana menebak apakah seorang pasien terkena penyakit jantung atau tidak menggunakan algoritma Decision Tree dan Naive Bayes
3. Algoritma mana yang akurasinya lebih tinggi pada dataset tersebut.

## 1.2 Tujuan Masalah

1. Tujuan melakukan EDA dan preprocessing pada dataset, melatih model Decision Tree dan Naive Bayes.
2. Menghitung akurasi keduanya, lalu membandingkan hasilnya.
3. Hasil yang diperoleh: akurasi Decision Tree sebesar 83,91% dan akurasi Naive Bayes sebesar 82,61% pada data uji.

## 1.3 Manfaat Masalah

1. Manfaat yang diperoleh adalah memahami alur kerja klasifikasi dari awal sampai akhir (membaca data, EDA, preprocessing, melatih model, menghitung akurasi).
2. Memahami perbedaan Decision Tree dan Naive Bayes.
3. Mengetahui faktor apa saja yang paling berkaitan dengan penyakit jantung pada data pasien.

# BAB II

# LANDASAN TEORI

## 2.1 Klasifikasi

Klasifikasi adalah salah satu cara pembelajaran mesin yang bertugas menebak kelas (kategori) dari sebuah data berdasarkan contoh data yang sudah diketahui jawabannya. Pada praktikum ini kelas yang ditebak adalah HeartDisease yang terdiri dari dua kelas, yaitu 0 (tidak terkena penyakit jantung) dan 1 (terkena penyakit jantung). Alur umumnya adalah memahami data (EDA), menyiapkan data (preprocessing), membagi data latih dan data uji, melatih model, lalu menghitung akurasi.

Dalam machine learning, klasifikasi termasuk ke dalam metode supervised learning atau pembelajaran terawasi. Metode ini menggunakan data yang sudah memiliki label sebagai bahan pembelajaran agar model dapat mengenali pola dari setiap kategori. Setelah proses pelatihan selesai, model akan menggunakan pola tersebut untuk memperkirakan kelas dari data baru yang belum pernah dipelajari sebelumnya. Oleh karena itu, kualitas data latih sangat berpengaruh terhadap kemampuan model dalam menghasilkan prediksi.

Penerapan klasifikasi dalam bidang kesehatan dapat membantu mengolah data pasien berdasarkan informasi seperti usia, jenis kelamin, tekanan darah, kadar kolesterol, detak jantung maksimum, dan hasil pemeriksaan lainnya. Informasi tersebut dapat digunakan untuk menemukan pola yang berkaitan dengan kondisi kesehatan pasien. Menurut Kumar et al. (2025), machine learning telah banyak digunakan dalam penelitian prediksi penyakit jantung karena mampu menganalisis data medis dan menemukan pola yang dapat membantu proses identifikasi penyakit.

Selain itu, keberhasilan klasifikasi tidak hanya ditentukan oleh algoritma yang digunakan, tetapi juga dipengaruhi oleh kualitas dataset, pemilihan fitur, serta metode evaluasi. Model yang memperoleh hasil baik pada data latih belum tentu memiliki performa yang sama ketika diterapkan pada data baru. Oleh karena itu, diperlukan pembagian data menjadi data latih dan data uji agar kemampuan model dapat dinilai secara lebih objektif. Penelitian Simanjuntak et al. (2025) juga membandingkan beberapa algoritma machine learning untuk klasifikasi penyakit jantung menggunakan dataset yang terdiri dari 918 sampel dan 12 fitur.

## 2.2 EDA dan Preprocessing

EDA (Exploratory Data Analysis) adalah tahap mengenali data sebelum dipakai untuk melatih model, misalnya melihat jumlah baris dan kolom, data kosong, data duplikat, sebaran kelas, serta hubungan antar kolom memakai grafik seperti countplot, boxplot, dan heatmap korelasi.

Preprocessing adalah tahap menyiapkan data agar bisa dipakai model. Pada praktikum ini preprocessing berupa mengganti nilai yang janggal (nilai 0 pada kolesterol dan tekanan darah) dengan nilai median, serta mengubah data huruf menjadi angka dengan LabelEncoder. Pada penelitian klasifikasi penyakit jantung, tahap persiapan dan pemilihan fitur terbukti mempengaruhi akurasi Naive Bayes (Azizah et al., 2023).

EDA memiliki peran penting karena dapat memberikan gambaran awal mengenai karakteristik dan kondisi dataset sebelum proses pemodelan dilakukan. Melalui EDA, peneliti dapat mengetahui apakah terdapat data yang tidak lengkap, nilai yang tidak wajar, data yang berulang, maupun perbedaan jumlah sampel pada setiap kelas. Visualisasi juga membantu melihat pola penyebaran data dan kemungkinan hubungan antara satu fitur dengan fitur lainnya. Dengan demikian, EDA dapat membantu menentukan langkah preprocessing yang sesuai dengan kondisi dataset.

Pada penelitian machine learning di bidang kesehatan, preprocessing menjadi salah satu tahapan yang perlu diperhatikan karena data medis dapat memiliki format dan rentang nilai yang

berbeda. Islam et al. (2024) membahas penggunaan berbagai teknik pengolahan data, seperti pembersihan data, penanganan nilai yang hilang, serta pengubahan data kategorikal menjadi bentuk numerik dalam penelitian prediksi penyakit jantung. Penelitian tersebut juga membahas pemilihan fitur untuk membantu menentukan variabel yang relevan dalam proses klasifikasi.

Penggantian nilai yang tidak wajar menggunakan median bertujuan agar data tetap dapat digunakan tanpa terlalu dipengaruhi oleh nilai ekstrem. Median dipilih karena nilainya berada di tengah setelah data diurutkan, sehingga relatif tidak mudah berubah akibat adanya nilai yang sangat tinggi atau rendah. Namun, penggantian nilai 0 perlu didasarkan pada pemeriksaan dataset terlebih dahulu untuk memastikan bahwa nilai tersebut memang menunjukkan data yang tidak tersedia atau tidak valid.

Sementara itu, LabelEncoder digunakan untuk mengubah data kategorikal yang awalnya berbentuk teks menjadi angka agar dapat diproses oleh algoritma machine learning. Contohnya, kategori jenis kelamin dan tipe nyeri dada diubah menjadi kode numerik. Hasil pengubahan tersebut membuat data lebih mudah digunakan dalam proses pelatihan model. Meskipun demikian, angka hasil encoding merupakan kode kategori dan tidak selalu menunjukkan tingkatan atau urutan tertentu.

**2.3 Decision Tree**

Decision Tree (pohon keputusan) adalah algoritma yang membuat aturan berbentuk pohon. Setiap simpul berisi pertanyaan tentang sebuah fitur (misalnya apakah ST_Slope lebih kecil dari 1,5?), setiap cabang adalah jawabannya, dan setiap daun adalah hasil tebakan kelas. Pohon memilih pertanyaan yang paling baik memisahkan kelas, diukur dengan gini atau entropy. Rumus entropy: $\text{Entropy} = - \Sigma\ p_i \log_2 p_i$, dengan $p_i$ adalah proporsi kelas ke-i pada simpul. Semakin kecil entropy, semakin murni simpul tersebut.

Kelebihan Decision Tree adalah mudah dipahami dan tidak perlu scaling. Kekurangannya, jika pohon dibiarkan terlalu dalam, model akan menghafal data latih sehingga akurasi data latih sangat tinggi tetapi akurasi data uji lebih rendah (overfitting). Karena itu kedalaman pohon bisa dibatasi dengan parameter max_depth.

Proses pembentukan Decision Tree dilakukan dengan membagi data secara bertahap berdasarkan fitur yang dianggap paling baik dalam memisahkan kelas. Pemilihan fitur tersebut dapat menggunakan perhitungan entropy dan information gain. Information gain digunakan untuk mengukur seberapa besar pengurangan ketidakmurnian data setelah dilakukan pemisahan berdasarkan suatu fitur. Fitur yang menghasilkan pengurangan entropy lebih besar akan lebih diprioritaskan untuk menjadi pemisah pada pohon keputusan.

Salah satu keunggulan Decision Tree adalah hasil klasifikasinya dapat ditampilkan dalam bentuk diagram pohon sehingga proses pengambilan keputusan lebih mudah dipahami. Setiap jalur dari simpul awal hingga simpul akhir menggambarkan serangkaian aturan yang digunakan model untuk menentukan hasil klasifikasi. Selain itu, Decision Tree dapat menghasilkan nilai feature importance yang menunjukkan kontribusi relatif setiap fitur dalam proses pengambilan keputusan model.

Untuk mengurangi kemungkinan overfitting, Decision Tree dapat dikendalikan melalui pengaturan parameter seperti max_depth, yaitu batas kedalaman pohon yang akan dibentuk. Selain itu, pruning dapat digunakan untuk menyederhanakan pohon dengan mengurangi bagian yang dianggap kurang memberikan manfaat terhadap kemampuan prediksi. Dengan struktur yang lebih

sederhana, model diharapkan dapat mempertahankan kemampuan klasifikasi pada data yang belum pernah digunakan dalam proses pelatihan.

Penelitian Wibowo et al. (2024) membahas penerapan beberapa algoritma machine learning untuk klasifikasi penyakit jantung, termasuk Decision Tree. Penelitian tersebut menggunakan dataset penyakit jantung dengan 1.025 data dan 14 atribut. Hasil penelitian menunjukkan bahwa Decision Tree memperoleh akurasi sebesar 93%. Penelitian tersebut memperlihatkan bahwa Decision Tree dapat digunakan untuk mengidentifikasi pola dalam data pasien dan menghasilkan klasifikasi berdasarkan atribut yang tersedia.

## 2.4 Naive Bayes

Naive Bayes adalah algoritma klasifikasi yang memakai teorema Bayes untuk menghitung peluang sebuah data termasuk ke kelas tertentu. Rumusnya: $P(C|X) = P(X|C) \times P(C) / P(X)$, dengan $P(C|X)$ peluang kelas C jika diketahui data X, $P(X|C)$ peluang data X muncul pada kelas C, dan $P(C)$ peluang awal kelas C. Model memilih kelas yang peluangnya paling besar.

Disebut 'naive' (naif) karena algoritma menganggap setiap fitur tidak saling berhubungan (independen) satu sama lain, padahal di data nyata fitur sering saling berkaitan. Meski asumsinya sederhana, Naive Bayes cepat dan sering memberi hasil yang baik. Varian GaussianNB dipakai untuk fitur berupa angka dengan menganggap sebaran tiap fitur mengikuti distribusi normal. Naive Bayes telah dipakai untuk mengklasifikasikan penderita penyakit jantung dengan akurasi 86,84% pada data UCI dengan pembagian data 75% latih dan 25% uji (Hirwono et al., 2023).

Dalam proses klasifikasi, Naive Bayes menghitung kemungkinan suatu data termasuk ke dalam masing-masing kelas berdasarkan informasi fitur yang tersedia. Perhitungan tersebut mempertimbangkan peluang awal suatu kelas dan peluang kemunculan fitur pada kelas tersebut. Setelah seluruh peluang dihitung, model akan menentukan kelas dengan nilai probabilitas paling tinggi sebagai hasil prediksi. Proses ini membuat Naive Bayes dapat melakukan klasifikasi dengan proses perhitungan yang relatif sederhana dan cepat.

Gaussian Naive Bayes merupakan salah satu varian Naive Bayes yang digunakan ketika fitur yang diolah berbentuk numerik. Algoritma ini menggunakan asumsi bahwa nilai setiap fitur pada masing-masing kelas mengikuti distribusi normal atau distribusi Gaussian. Berdasarkan asumsi tersebut, model menghitung peluang suatu nilai numerik muncul pada kelas tertentu. Contohnya, fitur usia, tekanan darah, kolesterol, dan detak jantung maksimum dapat digunakan sebagai informasi dalam memperkirakan kelas pasien.

Keunggulan Naive Bayes adalah proses perhitungannya relatif cepat dan tidak membutuhkan proses pelatihan yang terlalu kompleks. Algoritma ini juga dapat digunakan pada dataset dengan jumlah fitur yang cukup banyak. Namun, asumsi independensi antarfitur menjadi salah satu keterbatasannya karena dalam data medis beberapa variabel dapat memiliki hubungan satu sama lain. Oleh sebab itu, hasil klasifikasi Naive Bayes tetap perlu dievaluasi menggunakan data uji untuk mengetahui kemampuan model dalam menghasilkan prediksi.

Penelitian Bahri et al. (2025) membandingkan algoritma Decision Tree, Naive Bayes, dan Random Forest dalam menganalisis faktor risiko serangan jantung di Indonesia. Penelitian tersebut menggunakan beberapa rasio pembagian data, yaitu 70:30, 80:20, dan 90:10. Hasil penelitian menunjukkan bahwa Decision Tree dan Naive Bayes memperoleh akurasi tertinggi yang sama, yaitu 74,97% pada pengujian yang dilakukan. Hasil tersebut memperlihatkan bahwa kedua algoritma dapat

digunakan sebagai metode klasifikasi untuk menganalisis data yang berkaitan dengan penyakit jantung.

## 2.5 Evaluasi Model

Akurasi adalah persentase tebakan yang benar dibanding seluruh tebakan: Akurasi = jumlah tebakan benar / jumlah seluruh data uji. Confusion matrix adalah tabel yang menunjukkan jumlah tebakan benar dan salah untuk tiap kelas. Selain itu classification_report menampilkan precision (seberapa tepat tebakan suatu kelas), recall (seberapa banyak data suatu kelas yang berhasil ditemukan), dan f1-score (gabungan precision dan recall).

Evaluasi model merupakan tahapan penting untuk mengetahui seberapa baik algoritma dalam melakukan klasifikasi terhadap data yang diberikan. Pengujian dilakukan dengan membandingkan hasil prediksi model dengan label sebenarnya pada data uji. Melalui proses tersebut, dapat diketahui apakah model mampu mengenali pola yang telah dipelajari selama proses pelatihan dan menerapkannya pada data yang belum pernah digunakan sebelumnya.

# BAB III

# HASIL

## 3.1 Import library

Import library

[1]:

```
# 202531081_M. Fasha Bintang Pratama

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
```

- import numpy as np Digunakan untuk mengimpor library NumPy dengan nama singkat np. Library ini berfungsi untuk melakukan perhitungan numerik, pengolahan angka, serta operasi pada array.
- import pandas as pd Digunakan untuk mengimpor library Pandas dengan nama singkat pd. Library ini berfungsi untuk mengolah dan menganalisis data dalam bentuk tabel menggunakan DataFrame.
- import matplotlib.pyplot as plt Digunakan untuk mengimpor bagian pyplot dari Matplotlib dengan nama singkat plt. Library ini berfungsi untuk membuat visualisasi data seperti grafik garis, grafik batang, histogram, dan berbagai jenis grafik lainnya.
- import seaborn as sns Digunakan untuk mengimpor library Seaborn dengan nama singkat sns. Library ini berfungsi untuk membuat visualisasi data statistik dengan tampilan yang lebih menarik dan mudah dipahami.
- sns.set(style="whitegrid") Digunakan untuk mengatur tampilan grafik Seaborn menggunakan gaya *whitegrid*, yaitu latar belakang berwarna putih dengan garis-garis grid yang membantu mempermudah pembacaan nilai pada grafik.

## 3.2 Membaca dataset

[2]:

```
# 202531081_M. Fasha Bintang Pratama
df = pd.read_csv("01_heart_disease.csv")
df.head()
```

[2]:

| | Age | Sex | ChestPainType | RestingBP | Cholesterol | FastingBS | RestingECG | MaxHR | ExerciseAngina | Oldpeak | ST_Slope | HeartDisease |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 40 | M | ATA | 140 | 289 | 0 | Normal | 172 | N | 0.0 | Up | 0 |
| 1 | 49 | F | NAP | 160 | 180 | 0 | Normal | 156 | N | 1.0 | Flat | 1 |
| 2 | 37 | M | ATA | 130 | 283 | 0 | ST | 98 | N | 0.0 | Up | 0 |
| 3 | 48 | F | ASY | 138 | 214 | 0 | Normal | 108 | Y | 1.5 | Flat | 1 |
| 4 | 54 | M | NAP | 150 | 195 | 0 | Normal | 122 | N | 0.0 | Up | 0 |

- df = pd.read_csv("01_heart_disease.csv") Digunakan untuk membaca dan mengimpor data dari file CSV yang bernama 01_heart_disease.csv menggunakan library Pandas. Data yang telah dibaca kemudian disimpan ke dalam variabel df dalam bentuk DataFrame agar dapat diolah dan dianalisis lebih lanjut.
- df.head() Digunakan untuk menampilkan lima baris pertama dari DataFrame df. Fungsi ini bertujuan untuk melihat gambaran awal isi dataset, seperti nama kolom, nilai data, dan struktur tabel sebelum dilakukan proses analisis lebih lanjut.
- Dataset Heart Disease berisi informasi medis pasien yang mencakup data demografis, seperti Age (usia) dan Sex (jenis kelamin), serta berbagai indikator klinis yang digunakan untuk mendeteksi risiko penyakit jantung. Indikator tersebut meliputi ChestPainType (jenis nyeri dada), RestingBP (tekanan darah saat istirahat), Cholesterol (kadar kolesterol), dan FastingBS (kadar gula darah puasa). Selain itu, terdapat RestingECG (hasil pemeriksaan elektrokardiogram saat istirahat), MaxHR (detak jantung maksimum), ExerciseAngina (nyeri dada yang muncul saat berolahraga), Oldpeak (nilai depresi segmen ST akibat aktivitas fisik), dan ST_Slope (kemiringan segmen ST saat aktivitas mencapai puncak). Seluruh variabel tersebut digunakan untuk menganalisis kondisi kesehatan pasien dan memprediksi nilai pada kolom HeartDisease. Kolom HeartDisease merupakan variabel target yang menunjukkan apakah pasien tidak memiliki penyakit jantung (0) atau memiliki penyakit jantung (1).

## 3.3 Melihat informasi dataset (EDA)

Melihat informasi dataset (EDA)

[3]:
```
# 202531081_M. Fasha Bintang Pratama
df.info()
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    str
 2   ChestPainType   918 non-null    str
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    str
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    str
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    str
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), str(5)
memory usage: 86.2 KB
```

[3]:

| | Age | RestingBP | Cholesterol | FastingBS | MaxHR | Oldpeak | HeartDisease |
|---|---|---|---|---|---|---|---|
| count | 918.000000 | 918.000000 | 918.000000 | 918.000000 | 918.000000 | 918.000000 | 918.000000 |
| mean | 53.510893 | 132.396514 | 198.799564 | 0.233115 | 136.809368 | 0.887364 | 0.553377 |
| std | 9.432617 | 18.514154 | 109.384145 | 0.423046 | 25.460334 | 1.066570 | 0.497414 |
| min | 28.000000 | 0.000000 | 0.000000 | 0.000000 | 60.000000 | -2.600000 | 0.000000 |
| 25% | 47.000000 | 120.000000 | 173.250000 | 0.000000 | 120.000000 | 0.000000 | 0.000000 |
| 50% | 54.000000 | 130.000000 | 223.000000 | 0.000000 | 138.000000 | 0.600000 | 1.000000 |
| 75% | 60.000000 | 140.000000 | 267.000000 | 0.000000 | 156.000000 | 1.500000 | 1.000000 |
| max | 77.000000 | 200.000000 | 603.000000 | 1.000000 | 202.000000 | 6.200000 | 1.000000 |

- df.info() Digunakan untuk menampilkan informasi umum mengenai dataset yang tersimpan dalam variabel df. Informasi yang ditampilkan meliputi jumlah baris dan kolom, nama setiap kolom, jumlah data yang tidak kosong (*non-null*), tipe data, serta penggunaan memori. Fungsi ini membantu mengetahui struktur dataset dan memeriksa apakah terdapat data yang kosong.
- df.describe() Digunakan untuk menampilkan ringkasan statistik deskriptif dari dataset, khususnya pada kolom yang memiliki tipe data numerik. Informasi yang ditampilkan

meliputi jumlah data (*count*), nilai rata-rata (*mean*), standar deviasi (*std*), nilai minimum (*min*), kuartil pertama (25%), median (50%), kuartil ketiga (75%), dan nilai maksimum (*max*). Fungsi ini membantu memahami karakteristik dan persebaran data sebelum dilakukan analisis lebih lanjut.

- Dataset Heart Disease terdiri dari 918 sampel pasien dan 12 kolom yang memuat informasi mengenai kondisi kesehatan pasien. Berdasarkan hasil df.info(), seluruh 918 baris data terisi penuh tanpa adanya nilai yang kosong (*missing values*). Dataset ini menggunakan tipe data bilangan bulat (int64) pada kolom Age, RestingBP, Cholesterol, FastingBS, MaxHR, dan HeartDisease. Tipe data bilangan desimal (float64) digunakan pada kolom Oldpeak, sedangkan tipe data teks atau kategori (object) digunakan pada kolom Sex, ChestPainType, RestingECG, ExerciseAngina, dan ST_Slope.
- Hasil df.describe() menampilkan ringkasan statistik deskriptif dari tujuh variabel numerik dalam dataset. Berdasarkan hasil tersebut, rata-rata usia pasien adalah 53,5 tahun**,** dengan rentang usia antara 28 hingga 77 tahun. Rata-rata tekanan darah saat istirahat (RestingBP) sebesar 132,4 mmHg**,** sedangkan rata-rata kadar kolesterol (Cholesterol) sebesar 198,8 mg/dl**.** Terdapat nilai minimum 0 pada kedua variabel tersebut yang perlu diperiksa lebih lanjut karena kemungkinan menunjukkan data yang tidak valid. Kolom FastingBS memiliki nilai rata-rata 0,233**,** yang menunjukkan bahwa sekitar 23,3% pasien memiliki kadar gula darah puasa lebih dari 120 mg/dl. Selanjutnya, rata-rata detak jantung maksimum (MaxHR) adalah 136,8 bpm**,** sedangkan nilai rata-rata depresi segmen ST (Oldpeak) sebesar 0,88**.** Terakhir, kolom HeartDisease sebagai variabel target memiliki nilai rata-rata 0,553, yang menunjukkan bahwa sekitar 55,3% pasien dalam dataset termasuk dalam kategori memiliki penyakit jantung (1), sedangkan sisanya termasuk dalam kategori tidak memiliki penyakit jantung (0).

## 3.4 Cek data kosong, duplikat, dan nilai 0 yang janggal

**Cek data kosong, duplikat, dan nilai 0 yang janggal**

[4]:
```
# 202531081_M. Fasha Bintang Pratama
print("Data kosong:")
print(df.isnull().sum())
print("Data duplikat:", df.duplicated().sum())
print("Jumlah Cholesterol = 0 :", (df["Cholesterol"] == 0).sum())
print("Jumlah RestingBP   = 0 :", (df["RestingBP"] == 0).sum())
```

```
Data kosong:
Age               0
Sex               0
ChestPainType     0
RestingBP         0
Cholesterol       0
FastingBS         0
RestingECG        0
MaxHR             0
ExerciseAngina    0
Oldpeak           0
ST_Slope          0
HeartDisease      0
dtype: int64
Data duplikat: 0
Jumlah Cholesterol = 0 : 172
Jumlah RestingBP   = 0 : 1
```

- print("Data kosong:") Digunakan untuk menampilkan tulisan "Data kosong:" sebagai judul atau keterangan sebelum menampilkan hasil pemeriksaan data yang kosong.
- print(df.isnull().sum()) Digunakan untuk memeriksa dan menghitung jumlah nilai kosong (*missing values*) pada setiap kolom dalam dataset. Fungsi isnull() digunakan untuk

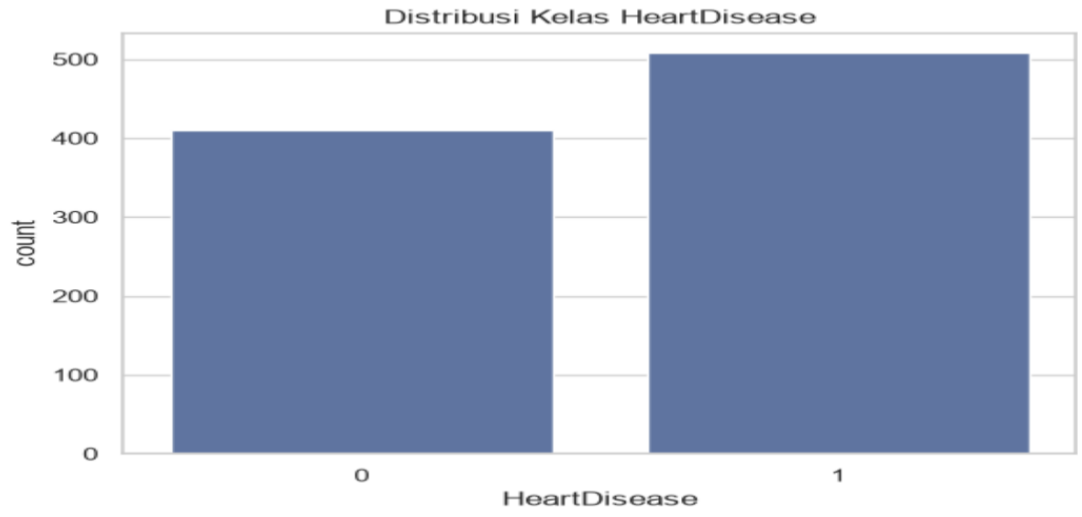
mendeteksi data yang kosong, sedangkan sum() digunakan untuk menjumlahkan seluruh data kosong pada masing-masing kolom.

- print("Data duplikat:", df.duplicated().sum()) Digunakan untuk menghitung jumlah baris data yang memiliki isi sama atau terduplikasi dalam dataset. Fungsi duplicated() digunakan untuk mendeteksi baris duplikat, sedangkan sum() digunakan untuk menghitung jumlah keseluruhannya.
- print("Jumlah Cholesterol = 0 :", (df["Cholesterol"] == 0).sum()) Digunakan untuk menghitung jumlah data pada kolom Cholesterol yang memiliki nilai 0. Pemeriksaan ini bertujuan untuk mengetahui apakah terdapat nilai kolesterol yang perlu diperiksa lebih lanjut karena kemungkinan menunjukkan data yang tidak valid.
- print("Jumlah RestingBP = 0 :", (df["RestingBP"] == 0).sum()) Digunakan untuk menghitung jumlah data pada kolom RestingBP yang memiliki nilai 0. Pemeriksaan ini bertujuan untuk mengidentifikasi nilai tekanan darah istirahat yang tidak valid dan perlu diperhatikan dalam proses pembersihan data.
- Berdasarkan hasil pemeriksaan kualitas data, dataset Heart Disease tidak memiliki nilai kosong (*missing values*) maupun baris duplikat. Hal ini ditunjukkan oleh seluruh 12 variabel yang memiliki nilai 0 pada pemeriksaan Data kosong serta jumlah Data duplikat sebanyak 0. Meskipun demikian, pemeriksaan lebih lanjut menemukan adanya nilai yang tidak sesuai dengan kondisi medis, yaitu nilai 0 pada kolom Cholesterol dan RestingBP. Terdapat sebanyak 172 sampel pasien yang memiliki kadar kolesterol sebesar 0 **(**Jumlah Cholesterol = 0 : 172**)** dan 1 sampel pasien yang memiliki tekanan darah saat istirahat sebesar 0 **(**Jumlah RestingBP = 0 : 1**).** Nilai tersebut perlu diperiksa lebih lanjut karena tidak mencerminkan kondisi medis yang wajar. Oleh karena itu, diperlukan tahap pembersihan data (*data preprocessing*), seperti melakukan imputasi (*imputation*) atau penanganan terhadap baris yang memiliki nilai tidak valid. Langkah ini bertujuan untuk meningkatkan kualitas dataset sebelum digunakan dalam proses analisis dan pemodelan machine learning.

## 3.5 Melihat sebaran kelas HeartDisease

Melihat sebaran kelas HeartDisease

```
[10]: # 202531081_M. Fasha Bintang Pratama
sns.countplot(x="HeartDisease", data=df)
plt.title("Distribusi Kelas HeartDisease")
plt.show()
print(df["HeartDisease"].value_counts())
```




```
HeartDisease
1    508
0    410
Name: count, dtype: int64
```

- sns.countplot(x="HeartDisease", data=df) Digunakan untuk membuat grafik batang (*count plot*) yang menampilkan jumlah data berdasarkan kategori pada kolom HeartDisease. Grafik ini menunjukkan perbandingan jumlah pasien yang tidak memiliki penyakit jantung (0) dan pasien yang memiliki penyakit jantung (1).
- plt.title("Distribusi Kelas HeartDisease") Digunakan untuk memberikan judul pada grafik, yaitu "Distribusi Kelas HeartDisease", agar grafik memiliki keterangan yang jelas mengenai informasi yang ditampilkan.
- plt.show() Digunakan untuk menampilkan grafik yang telah dibuat menggunakan Seaborn dan Matplotlib agar dapat dilihat dan dianalisis.
- print(df["HeartDisease"].value_counts()) Digunakan untuk menghitung dan menampilkan jumlah data pada setiap kategori dalam kolom HeartDisease. Fungsi value_counts() akan menunjukkan jumlah pasien dengan kategori 0 (tidak memiliki penyakit jantung) dan kategori 1 (memiliki penyakit jantung).
- Berdasarkan grafik dan hasil perhitungan, distribusi kelas pada variabel HeartDisease menunjukkan sebaran dari total 918 sampel pasien. Sebanyak 508 pasien (55,3%) termasuk dalam kategori memiliki penyakit jantung (kelas 1), sedangkan 410 pasien (44,7%) termasuk dalam kategori tidak memiliki penyakit jantung (kelas 0). Perbandingan jumlah kedua kategori tersebut menunjukkan bahwa distribusi kelas dalam dataset relatif seimbang (well-balanced dataset). Dengan demikian, dataset tidak mengalami ketimpangan kelas (class imbalance) yang signifikan, sehingga dapat digunakan untuk proses pelatihan model machine learning tanpa memerlukan penyeimbangan kelas secara khusus pada tahap awal.

## 3.6 Memisahkan fitur numerik dan kategorikal

### Memisahkan fitur numerik dan kategorikal

[11]:
```
# 202531081_M. Fasha Bintang Pratama
numerical_features = [f for f in df.columns if df[f].dtypes != "str"]
categorical_features = [f for f in df.columns if df[f].dtypes == "str"]

print("jumlah fitur numerik =", len(numerical_features))
print("daftar fitur numerik =", numerical_features)
print("jumlah fitur kategorikal =", len(categorical_features))
print("daftar fitur kategorikal =", categorical_features)

for i in categorical_features:
    print(i, ":", df[i].unique())
```

```
jumlah fitur numerik = 7
daftar fitur numerik = ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak', 'HeartDisease']
jumlah fitur kategorikal = 5
daftar fitur kategorikal = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
Sex : <StringArray>
['M', 'F']
Length: 2, dtype: str
ChestPainType : <StringArray>
['ATA', 'NAP', 'ASY', 'TA']
Length: 4, dtype: str
RestingECG : <StringArray>
['Normal', 'ST', 'LVH']
Length: 3, dtype: str
ExerciseAngina : <StringArray>
['N', 'Y']
Length: 2, dtype: str
ST_Slope : <StringArray>
['Up', 'Flat', 'Down']
Length: 3, dtype: str
```

- numerical_features = [f for f in df.columns if df[f].dtypes != "str"] Digunakan untuk mengelompokkan kolom yang memiliki tipe data selain string ke dalam variabel numerical_features. Pengelompokan ini bertujuan untuk mengidentifikasi fitur numerik yang dapat digunakan dalam proses analisis data.
- categorical_features = [f for f in df.columns if df[f].dtypes == "str"] Digunakan untuk mengelompokkan kolom yang memiliki tipe data string ke dalam variabel categorical_features.Fitur kategorikal merupakan data yang berisi kategori atau kelompok tertentu dan bukan berupa nilai perhitungan numerik.
- print("jumlah fitur numerik =", len(numerical_features)) Digunakan untuk menampilkan jumlah seluruh fitur numerik yang telah dikelompokkan sebelumnya. Fungsi len()digunakan untuk menghitung banyaknya kolom dalam variabel numerical_features.
- print("daftar fitur numerik =", numerical_features) Digunakan untuk menampilkan daftar nama kolom yang termasuk ke dalam kelompok fitur numerik.
- print("jumlah fitur kategorikal =", len(categorical_features)) Digunakan untuk menampilkan jumlah seluruh fitur kategorikal yang telah dikelompokkan.

Fungsi len() digunakan untuk menghitung banyaknya kolom dalam variabel categorical_features.

- print("daftar fitur kategorikal =", categorical_features) Digunakan untuk menampilkan daftar nama kolom yang termasuk ke dalam kelompok fitur kategorikal.
- for i in categorical_features: Merupakan perulangan yang digunakan untuk memeriksa setiap kolom yang termasuk dalam kelompok fitur kategorikal. Variabel i berfungsi untuk mewakili nama kolom yang sedang diperiksa dalam setiap perulangan.
- print(i, ":", df[i].unique()) Digunakan untuk menampilkan nama setiap kolom kategorikal beserta seluruh nilai unik yang terdapat di dalamnya. Fungsi unique() bertujuan untuk mengetahui berbagai kategori berbeda yang dimiliki oleh setiap kolom tanpa menampilkan nilai yang berulang.
- Pengelompokan jenis variabel pada dataset Heart Disease yang terdiri dari 7 fitur numerik dan 5 fitur kategorikal. Fitur numerik meliputi Age, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak, dan HeartDisease. Sementara itu, fitur kategorikal terdiri dari Sex, ChestPainType, RestingECG, ExerciseAngina, dan ST_Slope. Berdasarkan hasil pemeriksaan nilai unik, kolom Sex memiliki 2 kategori, yaitu M dan F. Kolom ChestPainType memiliki 4 kategori, yaitu ATA, NAP, ASY, dan TA. Kolom RestingECG memiliki 3 kategori, yaitu Normal, ST, dan LVH. Selanjutnya, kolom ExerciseAngina memiliki 2 kategori, yaitu N dan Y, sedangkan kolom ST_Slope memiliki 3 kategori, yaitu Up, Flat, dan Down. Identifikasi jenis variabel dan nilai unik pada setiap fitur kategorikal ini bertujuan untuk memahami karakteristik data serta menentukan metode pengolahan yang sesuai. Tahap ini juga menjadi persiapan sebelum melakukan proses encoding, seperti One-Hot Encoding atau Label Encoding, agar data kategorikal dapat diproses oleh model machine learning.

## 3.7 Grafik fitur kategorikal terhadap HeartDisease

```
## Grafik fitur kategorikal terhadap HeartDisease
```

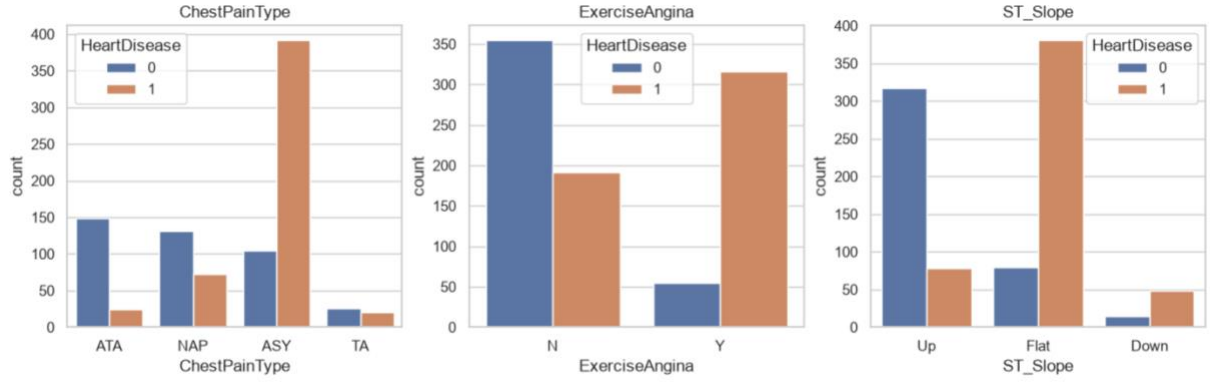
```
[12]: # 202531081_M. Fasha Bintang Pratama
plt.figure(figsize=(15, 4))

plt.subplot(1, 3, 1)
sns.countplot(x="ChestPainType", hue="HeartDisease", data=df)
plt.title("ChestPainType")

plt.subplot(1, 3, 2)
sns.countplot(x="ExerciseAngina", hue="HeartDisease", data=df)
plt.title("ExerciseAngina")

plt.subplot(1, 3, 3)
sns.countplot(x="ST_Slope", hue="HeartDisease", data=df)
plt.title("ST_Slope")
```

```
[12]: Text(0.5, 1.0, 'ST_Slope')
```



- plt.figure(figsize=(15, 4)) Digunakan untuk membuat area gambar atau kanvas grafik dengan ukuran lebar 15 inci dan tinggi 4 inci. Pengaturan ini bertujuan agar ketiga grafik dapat ditampilkan secara berdampingan dengan ukuran yang sesuai.
- plt.subplot(1, 3, 1) Digunakan untuk membagi area gambar menjadi 1 baris dan 3 kolom, kemudian memilih posisi grafik pertama. Pembagian ini memungkinkan beberapa grafik ditampilkan dalam satu tampilan.
- sns.countplot(x="ChestPainType", hue="HeartDisease", data=df) Digunakan untuk membuat grafik batang yang menampilkan jumlah pasien berdasarkan jenis nyeri dada (ChestPainType), dengan membedakan warna batang berdasarkan kategori HeartDisease. Grafik ini bertujuan untuk melihat hubungan antara jenis nyeri dada dengan kondisi penyakit jantung pasien.
- plt.title("ChestPainType") Digunakan untuk memberikan judul pada grafik pertama, yaitu ChestPainType.
- plt.subplot(1, 3, 2) Digunakan untuk memilih posisi grafik kedua pada area gambar, yaitu bagian tengah dari tiga kolom yang telah disediakan.
- sns.countplot(x="ExerciseAngina", hue="HeartDisease", data=df) Digunakan untuk membuat grafik batang yang menampilkan jumlah pasien berdasarkan kondisi ExerciseAngina, yaitu ada atau tidaknya nyeri dada saat melakukan aktivitas fisik. Warna batang dibedakan berdasarkan kategori HeartDisease untuk melihat perbandingan kondisi pasien.
- plt.title("ExerciseAngina") Digunakan untuk memberikan judul pada grafik kedua, yaitu ExerciseAngina.
- plt.subplot(1, 3, 3) Digunakan untuk memilih posisi grafik ketiga pada area gambar, yaitu bagian paling kanan dari tiga kolom yang tersedia.
- sns.countplot(x="ST_Slope", hue="HeartDisease", data=df) Digunakan untuk membuat grafik batang yang menampilkan jumlah pasien berdasarkan kategori kemiringan segmen ST (ST_Slope). Warna batang dibedakan berdasarkan kategori HeartDisease untuk mengetahui perbandingan jumlah pasien yang memiliki dan tidak memiliki penyakit jantung pada setiap kategori.
- plt.title("ST_Slope") Digunakan untuk memberikan judul pada grafik ketiga, yaitu ST_Slope.
- Gambar ini menampilkan grafik countplot yang menggambarkan hubungan antara tiga variabel kategorikal, yaitu ChestPainType, ExerciseAngina, dan ST_Slope, dengan variabel target HeartDisease. Kategori HeartDisease terdiri dari nilai 0 yang menunjukkan pasien tidak memiliki penyakit jantung dan nilai 1 yang menunjukkan pasien memiliki penyakit jantung.

  Berdasarkan grafik ChestPainType, pasien dengan jenis nyeri dada Asymptomatic (ASY) memiliki jumlah penderita penyakit jantung yang lebih dominan dibandingkan pasien yang tidak memiliki penyakit jantung. Sementara itu, pada kategori Atypical Angina (ATA) dan Non-Anginal Pain (NAP), jumlah pasien tanpa penyakit jantung terlihat lebih dominan. Pada grafik ExerciseAngina, pasien yang mengalami nyeri dada saat melakukan aktivitas fisik (Y) memiliki jumlah penderita penyakit jantung yang lebih tinggi dibandingkan pasien yang tidak mengalami nyeri dada (N). Selanjutnya, pada grafik ST_Slope, kategori Flat menunjukkan jumlah penderita penyakit jantung yang lebih dominan, diikuti oleh kategori Down. Sebaliknya, kategori Up lebih banyak ditemukan pada pasien yang tidak

memiliki penyakit jantung. Hasil visualisasi ini menunjukkan bahwa ketiga variabel kategorikal tersebut memiliki pola distribusi yang berbeda terhadap kondisi penyakit jantung dan dapat digunakan sebagai informasi awal dalam proses analisis serta pemodelan machine learning.

## 3.8 Boxplot fitur numerik terhadap HeartDisease

Boxplot fitur numerik terhadap HeartDisease

```
[13]: # 202531081_M. Fasha Bintang Pratama
plt.figure(figsize=(15, 4))

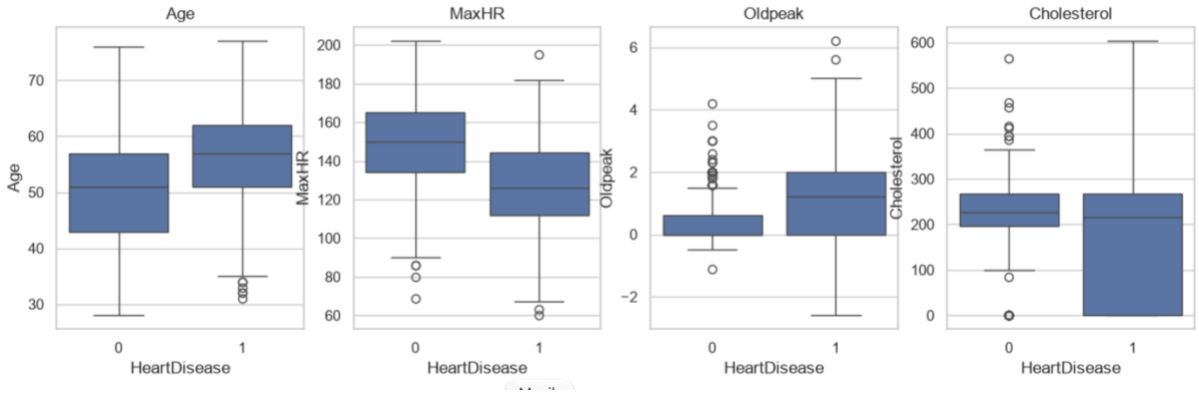
plt.subplot(1, 4, 1)
sns.boxplot(x="HeartDisease", y="Age", data=df)
plt.title("Age")

plt.subplot(1, 4, 2)
sns.boxplot(x="HeartDisease", y="MaxHR", data=df)
plt.title("MaxHR")

plt.subplot(1, 4, 3)
sns.boxplot(x="HeartDisease", y="Oldpeak", data=df)
plt.title("Oldpeak")

plt.subplot(1, 4, 4)
sns.boxplot(x="HeartDisease", y="Cholesterol", data=df)
plt.title("Cholesterol")
```

```
[13]: Text(0.5, 1.0, 'Cholesterol')
```




- plt.figure(figsize=(15, 4)) digunakan untuk membuat area gambar dengan ukuran lebar 15 inci dan tinggi 4 inci agar keempat grafik dapat ditampilkan dengan rapi.
- plt.subplot(1, 4, 1) digunakan untuk membagi area gambar menjadi 1 baris dan 4 kolom, kemudian memilih posisi grafik pertama.
- sns.boxplot(x="HeartDisease", y="Age", data=df) digunakan untuk membuat boxplot yang membandingkan distribusi usia pasien berdasarkan kondisi penyakit jantung. Nilai 0 menunjukkan pasien tidak memiliki penyakit jantung, sedangkan nilai 1 menunjukkan pasien memiliki penyakit jantung.
- plt.title("Age") digunakan untuk memberikan judul "Age" pada grafik pertama.
- plt.subplot(1, 4, 2) digunakan untuk memilih posisi grafik kedua dari empat bagian yang telah dibuat.
- sns.boxplot(x="HeartDisease", y="MaxHR", data=df) digunakan untuk menampilkan distribusi detak jantung maksimum pasien berdasarkan kondisi penyakit jantung.
- plt.title("MaxHR") digunakan untuk memberikan judul "MaxHR" pada grafik kedua.
- plt.subplot(1, 4, 3) digunakan untuk memilih posisi grafik ketiga.
- sns.boxplot(x="HeartDisease", y="Oldpeak", data=df) digunakan untuk menampilkan distribusi nilai Oldpeak berdasarkan kondisi penyakit jantung. Oldpeak merupakan ukuran

perubahan segmen ST pada hasil pemeriksaan elektrokardiogram (EKG) saat pasien melakukan aktivitas fisik dibandingkan saat istirahat.

- plt.title("Oldpeak") digunakan untuk memberikan judul "Oldpeak" pada grafik ketiga.
- plt.subplot(1, 4, 4) digunakan untuk memilih posisi grafik keempat.
- sns.boxplot(x="HeartDisease", y="Cholesterol", data=df) digunakan untuk menampilkan distribusi kadar kolesterol pasien berdasarkan kondisi penyakit jantung.
- plt.title("Cholesterol") digunakan untuk memberikan judul "Cholesterol" pada grafik keempat.
- Visualisasi boxplot ini menunjukkan perbandingan distribusi empat variabel numerik, yaitu Age, MaxHR, Oldpeak, dan Cholesterol, berdasarkan status HeartDisease (0 = tidak memiliki penyakit jantung, 1 = memiliki penyakit jantung). Berdasarkan grafik, pasien dengan penyakit jantung secara umum cenderung memiliki usia yang lebih tinggi, detak jantung maksimum (MaxHR) yang lebih rendah, serta nilai Oldpeak yang lebih tinggi dibandingkan pasien tanpa penyakit jantung. Sementara itu, variabel Cholesterol memiliki sebaran data yang cukup luas dan terdapat nilai 0 yang perlu diperiksa lebih lanjut karena kemungkinan merupakan data yang tidak valid atau belum tercatat. Selain itu, boxplot juga memperlihatkan median, rentang penyebaran data, dan nilai pencilan (outlier) pada setiap variabel. Hasil visualisasi ini memberikan gambaran awal mengenai perbedaan karakteristik pasien berdsarkan status penyakit jantung.

### 3.9 Preprocessing: mengganti nilai 0 dengan median

```
## Preprocessing: mengganti nilai 0 dengan median
```

[14]:
```
# 202531081_M. Fasha Bintang Pratama
for kolom in ["Cholesterol", "RestingBP"]:
    median = df.loc[df[kolom] != 0, kolom].median()
    df[kolom] = df[kolom].replace(0, median)
    print(kolom, "-> nilai 0 diganti median", median)

print("Cholesterol minimum sekarang:", df["Cholesterol"].min())
print("RestingBP minimum sekarang  :", df["RestingBP"].min())
```

```
Cholesterol -> nilai 0 diganti median 237.0
RestingBP -> nilai 0 diganti median 130.0
Cholesterol minimum sekarang: 85
RestingBP minimum sekarang  : 80
```

- for kolom in ["Cholesterol", "RestingBP"]: digunakan untuk melakukan perulangan pada dua kolom, yaitu Cholesterol dan RestingBP, sehingga proses pengolahan data dapat dilakukan secara otomatis pada kedua kolom tersebut.
- median = df.loc[df[kolom] != 0, kolom].median() digunakan untuk menghitung nilai median dari setiap kolom dengan hanya mengambil data yang tidak bernilai 0. Nilai 0 tidak disertakan agar tidak memengaruhi perhitungan median.
- df[kolom] = df[kolom].replace(0, median) digunakan untuk mengganti seluruh nilai 0 pada kolom yang sedang diproses dengan nilai median yang telah dihitung sebelumnya.
- print(kolom, "-> nilai 0 diganti median", median) digunakan untuk menampilkan nama kolom dan nilai median yang digunakan sebagai pengganti nilai 0.
- print("Cholesterol minimum sekarang:", df["Cholesterol"].min()) digunakan untuk menampilkan nilai minimum terbaru pada kolom Cholesterol setelah proses penggantian nilai 0 selesai dilakukan.
- print("RestingBP minimum sekarang :", df["RestingBP"].min()) digunakan untuk menampilkan nilai minimum terbaru pada kolom RestingBP untuk memastikan bahwa nilai 0 telah diganti.
- Hasil menunjukkan bahwa proses preprocessing untuk menangani nilai 0 pada variabel Cholesterol dan RestingBP telah berhasil dilakukan. Nilai 0 pada variabel Cholesterol diganti menggunakan median dari data non-nol sebesar 237.0, sedangkan pada variabel RestingBP diganti dengan median sebesar 130.0. Setelah proses imputasi selesai, nilai minimum Cholesterol berubah menjadi 85 dan RestingBP menjadi 80. Hasil ini menunjukkan bahwa seluruh nilai 0 pada kedua variabel tersebut telah berhasil diganti. Dengan demikian, data menjadi lebih siap untuk digunakan pada tahap analisis dan pemodelan machine learning, meskipun tetap perlu dilakukan pemeriksaan kualitas data lebih lanjut.

## 3.10 Preprocessing: mengubah data huruf menjadi angka

Preprocessing: mengubah data huruf menjadi angka

[10]:
```
# 202531081_M. Fasha Bintang Pratama
from sklearn.preprocessing import LabelEncoder

for feature in categorical_features:
    le = LabelEncoder()
    df[feature] = le.fit_transform(df[feature])
    print(feature, ":", {label: i for i, label in enumerate(le.classes_)})

df.head()
```

```
Sex : {'F': 0, 'M': 1}
ChestPainType : {'ASY': 0, 'ATA': 1, 'NAP': 2, 'TA': 3}
RestingECG : {'LVH': 0, 'Normal': 1, 'ST': 2}
ExerciseAngina : {'N': 0, 'Y': 1}
ST_Slope : {'Down': 0, 'Flat': 1, 'Up': 2}
```

[10]:

| | Age | Sex | ChestPainType | RestingBP | Cholesterol | FastingBS | RestingECG | MaxHR | ExerciseAngina | Oldpeak | ST_Slope | HeartDisease |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 40 | 1 | 1 | 140 | 289 | 0 | 1 | 172 | 0 | 0.0 | 2 | 0 |
| 1 | 49 | 0 | 2 | 160 | 180 | 0 | 1 | 156 | 0 | 1.0 | 1 | 1 |
| 2 | 37 | 1 | 1 | 130 | 283 | 0 | 2 | 98 | 0 | 0.0 | 2 | 0 |
| 3 | 48 | 0 | 0 | 138 | 214 | 0 | 1 | 108 | 1 | 1.5 | 1 | 1 |
| 4 | 54 | 1 | 2 | 150 | 195 | 0 | 1 | 122 | 0 | 0.0 | 2 | 0 |

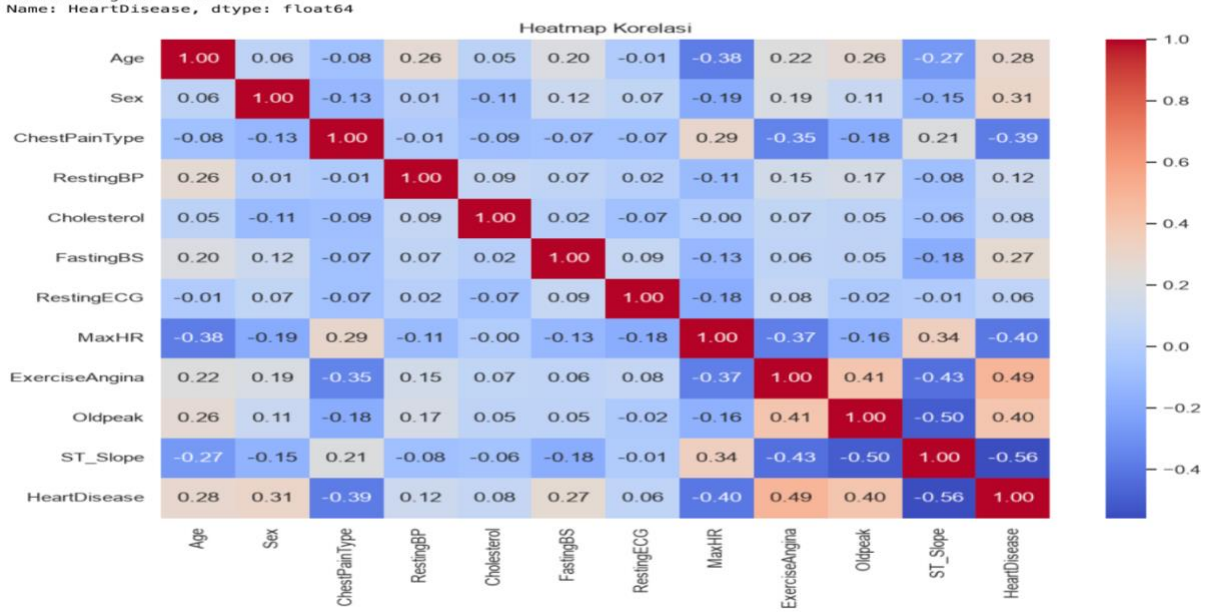
- from sklearn.preprocessing import LabelEncoder digunakan untuk mengimpor LabelEncoder dari library Scikit-learn. Fungsi ini digunakan untuk mengubah data kategorikal atau berbentuk teks menjadi angka agar dapat diproses oleh algoritma machine learning.

- for feature in categorical_feataures: digunakan untuk melakukan perulangan pada setiap fitur kategorikal yang sebelumnya telah dikumpulkan dalam variabel categorical_features.
- le = LabelEncoder() digunakan untuk membuat objek LabelEncoder yang bertugas mengubah setiap kategori menjadi nilai numerik.
- df[feature] = le.fit_transform(df[feature]) digunakan untuk mempelajari kategori unik pada kolom yang sedang diproses, kemudian mengubah setiap kategori menjadi angka dan menyimpan hasilnya kembali ke dalam DataFrame df.
- print(feature, ":", {label: i for i, label in enumerate(le.classes_)}) digunakan untuk menampilkan nama fitur beserta pemetaan setiap kategori ke angka hasil encoding. Dengan demikian, dapat diketahui angka yang mewakili masing-masing kategori.
- df.head() digunakan untuk menampilkan lima baris pertama dataset setelah proses Label Encoding, sehingga hasil perubahan data kategorikal menjadi numerik dapat diperiksa.
- Gambar tersebut menampilkan lima baris pertama dari DataFrame setelah proses encoding, yaitu pengubahan variabel kategorikal menjadi angka numerik menggunakan metode Label Encoding. Variabel yang sebelumnya berbentuk teks, seperti Sex, ChestPainType, RestingECG, ExerciseAngina, dan ST_Slope, kini telah direpresentasikan dalam bentuk angka. Sebagai contoh, variabel Sex diubah menjadi 0 untuk Female dan 1 untuk Male, sedangkan ExerciseAngina diubah menjadi 0 untuk N (No) dan 1 untuk Y (Yes). Sementara itu, variabel ChestPainType, RestingECG, dan ST_Slope juga diberikan kode angka sesuai urutan kategori yang dipelajari oleh LabelEncoder. Proses encoding ini bertujuan agar seluruh fitur dapat direpresentasikan dalam bentuk numerik sehingga lebih mudah diproses oleh algoritma machine learning pada tahap pemodelan.

## 3.11 Heatmap korelasi

```
[11]: # 202531081_M. Fasha Bintang Pratama
      plt.figure(figsize=(11, 8))
      sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
      plt.title("Heatmap Korelasi")
      print(df.corr()["HeartDisease"].drop("HeartDisease").sort_values())
```

```
ST_Slope          -0.558771
MaxHR             -0.400421
ChestPainType     -0.386828
RestingECG         0.057384
Cholesterol        0.076114
RestingBP          0.117798
FastingBS          0.267291
Age                0.282039
Sex                0.305445
Oldpeak            0.403951
ExerciseAngina     0.494282
Name: HeartDisease, dtype: float64
```



- plt.figure(figsize=(11, 8)) digunakan untuk membuat area gambar dengan ukuran lebar 11 inci dan tinggi 8 inci agar heatmap dapat ditampilkan dengan jelas.
- sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm") digunakan untuk membuat visualisasi heatmap berdasarkan matriks korelasi antarvariabel numerik dalam dataset. Parameter df.corr() menghitung nilai korelasi, annot=True menampilkan angka pada setiap kotak, fmt=".2f" mengatur angka agar ditampilkan dengan dua angka di belakang koma, sedangkan cmap="coolwarm" menentukan gradasi warna untuk membedakan nilai korelasi positif dan negatif.
- plt.title("Heatmap Korelasi") digunakan untuk memberikan judul "Heatmap Korelasi" pada visualisasi.
- print(df.corr()["HeartDisease"].drop("HeartDisease").sort_values()) digunakan untuk menampilkan nilai korelasi setiap variabel terhadap HeartDisease. Bagian df.corr()["HeartDisease"] mengambil korelasi seluruh variabel dengan HeartDisease, drop("HeartDisease") menghapus korelasi variabel dengan dirinya sendiri,sedangkan sort_values() mengurutkan hasil korelasi dari nilai terkecil hingga terbesar.
- Visualisasi Heatmap Korelasi dan hasil perhitungan nilai korelasi menunjukkan tingkat hubungan linear antara setiap variabel fitur dengan variabel target HeartDisease. Berdasarkan hasil tersebut, ExerciseAngina (0,49), Oldpeak (0,40), Sex (0,31), Age (0,28), dan FastingBS (0,27) memiliki korelasi positif, yang menunjukkan bahwa peningkatan nilai variabel tersebut cenderung berkaitan dengan meningkatnya nilai HeartDisease. Sebaliknya, ST_Slope (-0,56), MaxHR (-0,40), dan ChestPainType (-0,39) menunjukkan korelasi negatif, yang berarti peningkatan nilai hasil encoding variabel tersebut cenderung berkaitan dengan penurunan nilai HeartDisease. Sementara itu, RestingBP (0,12), Cholesterol (0,08), dan RestingECG (0,06) memiliki korelasi linear yang relatif lemah terhadap HeartDisease. Hasil analisis ini memberikan gambaran awal mengenai variabel yang memiliki hubungan paling kuat maupun paling lemah dengan status penyakit jantung. Namun, nilai korelasi tidak membuktikan hubungan sebab-akibat dan perlu ditafsirkan dengan memperhatikan pengkodean pada setiap variabel kategorikal.

## 3.12 Memisahkan fitur (x) dan target (y)




- x = df.drop("HeartDisease", axis=1) digunakan untuk memisahkan seluruh kolom fitur dari dataset dengan menghapus kolom HeartDisease. Variabel x berisi data yang akan

digunakan sebagai input atau variabel prediktor dalam proses pemodelan machine learning. Parameter axis=1 menunjukkan bahwa penghapusan dilakukan pada kolom.

- y = df["HeartDisease"] digunakan untuk mengambil kolom HeartDisease sebagai variabel target atau label yang akan diprediksi oleh model machine learning.
- print("Bentuk x:", x.shape) digunakan untuk menampilkan ukuran atau dimensi data x, yaitu jumlah baris dan jumlah kolom fitur yang digunakan sebagai input model.
- print("Bentuk y:", y.shape) digunakan untuk menampilkan ukuran atau dimensi data y, yaitu jumlah baris pada variabel target HeartDisease.
- Keluaran Bentuk x: (918, 11) dan Bentuk y: (918,) menunjukkan bahwa dataset berhasil dipisahkan menjadi matriks fitur (x) dan variabel target (y). Matriks x memiliki dimensi 918 baris dan 11 kolom, yang berarti terdapat 918 data pasien dengan 11 atribut medis yang digunakan sebagai variabel prediktor. Sementara itu, variabel y memiliki 918 baris dalam bentuk array satu dimensi yang berisi label status penyakit jantung untuk setiap pasien, yaitu 0 (tidak memiliki penyakit jantung) dan 1 (memiliki penyakit jantung). Hasil ini menunjukkan bahwa jumlah data pada variabel fitur dan target sudah sesuai sehingga dapat dilanjutkan ke tahap pembagian data latih dan data uji (train-test split).

## 3.13 Membagi data latih dan data uji

### Membagi data latih dan data uji

13]:
```
# 202531081_M. Fasha Bintang Pratama
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=0)
print("x_train:", x_train.shape)
print("x_test :", x_test.shape)
```

```
x_train: (688, 11)
x_test : (230, 11)
```

- from sklearn.model_selection import train_test_split digunakan untuk mengimpor fungsi train_test_split dari library Scikit-learn. Fungsi ini digunakan untuk membagi dataset menjadi data latih (training data) dan data uji (testing data)
- x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=0) digunakan untuk membagi data fitur (x) dan target (y) menjadi empat bagian. Parameter test_size=0.25 berarti 25% data digunakan sebagai data uji, sedangkan 75% sisanya digunakan sebagai data latih. Parameter random_state=0 digunakan agar pembagian data menghasilkan susunan yang konsisten ketika program dijalankan kembali dengan dataset dan pengaturan yang sama.
- print("x_train:", x_train.shape) digunakan untuk menampilkan ukuran data fitur yang digunakan sebagai data latih, yaitu jumlah baris dan jumlah kolom.
- print("x_test :", x_test.shape) digunakan untuk menampilkan ukuran datafitur yang digunakan sebagai data uji, yaitu jumlah baris dan jumlah kolom.
- Keluaran x_train: (688, 11) dan x_test: (230, 11) menunjukkan hasil pembagian dataset menjadi dua bagian, yaitu data latih (training set) dan data uji (testing set). Dari total 918 sampel, sebanyak 688 baris data atau sekitar 75% digunakan sebagai data latih untuk membangun model machine learning, sedangkan 230 baris data atau sekitar 25% digunakan sebagai data uji untuk mengevaluasi kemampuan model dalam memprediksi

data yang belum digunakan selama proses pelatihan. Kedua subset tersebut tetap memiliki 11 atribut medis sebagai variabel prediktor. Hasil ini menunjukkan bahwa dataset telah terbagi sesuai dengan proporsi yang ditentukan dan siap digunakan dalam proses pelatihan serta pengujian model.

## 3.14 Decision Tree - model pertama

### Decision Tree - model pertama (pengaturan bawaan)

[14]:
```
# 202531081_M. Fasha Bintang Pratama
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

dt1 = DecisionTreeClassifier(random_state=0)
dt1.fit(x_train, y_train)

print("Kedalaman pohon:", dt1.get_depth())
print("Akurasi data latih:", accuracy_score(y_train, dt1.predict(x_train)))
print("Akurasi data uji  :", accuracy_score(y_test, dt1.predict(x_test)))
```

```
Kedalaman pohon: 13
Akurasi data latih: 1.0
Akurasi data uji  : 0.7695652173913043
```

- from sklearn.tree import DecisionTreeClassifier digunakan untuk mengimpor algoritma Decision Tree Classifier dari library Scikit-learn. Algoritma ini digunakan untuk membangun model klasifikasi berdasarkan pola dan aturan keputusan dari data.
- from sklearn.metrics import accuracy_score digunakan untuk mengimpor fungsi accuracy_score yang berfungsi menghitung tingkat akurasi model dalam memprediksi data dengan membandingkan hasil prediksi dan label sebenarnya.
- dt1 = DecisionTreeClassifier(random_state=0) digunakan untuk membuat objek model Decision Tree Classifier. Parameter random_state=0 digunakan agar proses yang melibatkan unsur acak menghasilkan hasil yang konsisten ketika program dijalankan kembali dengan data dan pengaturan yang sama.
- dt1.fit(x_train, y_train) digunakan untuk melatih model menggunakan data latih. Variabel x_train berisi fitur atau atribut pasien, sedangkan y_train berisi label status penyakit jantung yang menjadi target prediksi.
- print("Kedalaman pohon:", dt1.get_depth()) digunakan untuk menampilkan kedalaman pohon keputusan yang telah terbentuk selama proses pelatihan. Kedalaman menunjukkan panjang jalur terpanjang dari akar pohon hingga simpul terakhir.
- print("Akurasi data latih:", accuracy_score(y_train, dt1.predict(x_train))) digunakan untuk menghitung dan menampilkan akurasi model pada data latih. Fungsi dt1.predict(x_train) menghasilkan prediksi berdasarkan data latih, kemudian hasilnya dibandingkan dengan label sebenarnya pada y_train.
- print("Akurasi data uji :", accuracy_score(y_test, dt1.predict(x_test))) digunakan untuk menghitung dan menampilkan akurasi model pada data uji. Hasil prediksi dari x_test dibandingkan dengan label sebenarnya pada y_test untuk mengetahui kemampuan model dalam memprediksi data yang tidak digunakan selama pelatihan.
- Hasil menyajikan performa awal dari model *Decision Tree* tanpa pembatasan kedalaman pohon (*unconstrained tree*). Pohon keputusan yang terbentuk memiliki Kedalaman pohon: 13, yang menghasilkan Akurasi data latih sebesar 1.0 (100%) dan Akurasi data uji sebesar

0.7696 (76,96%). Perbedaan yang cukup signifikan antara akurasi data latih yang sempurna dan akurasi data uji yang lebih rendah ini mengindikasikan terjadinya kondisi *overfitting*, di mana model menghafal pola data latih secara terlalu mendalam hingga ke tingkat *noise*, sehingga performanya menurun ketika diuji menggunakan data baru yang belum pernah dilihat sebelumnya.

## 3.15 Decision Tree - mencoba beberapa max_depth

Decision Tree - mencoba beberapa max_depth

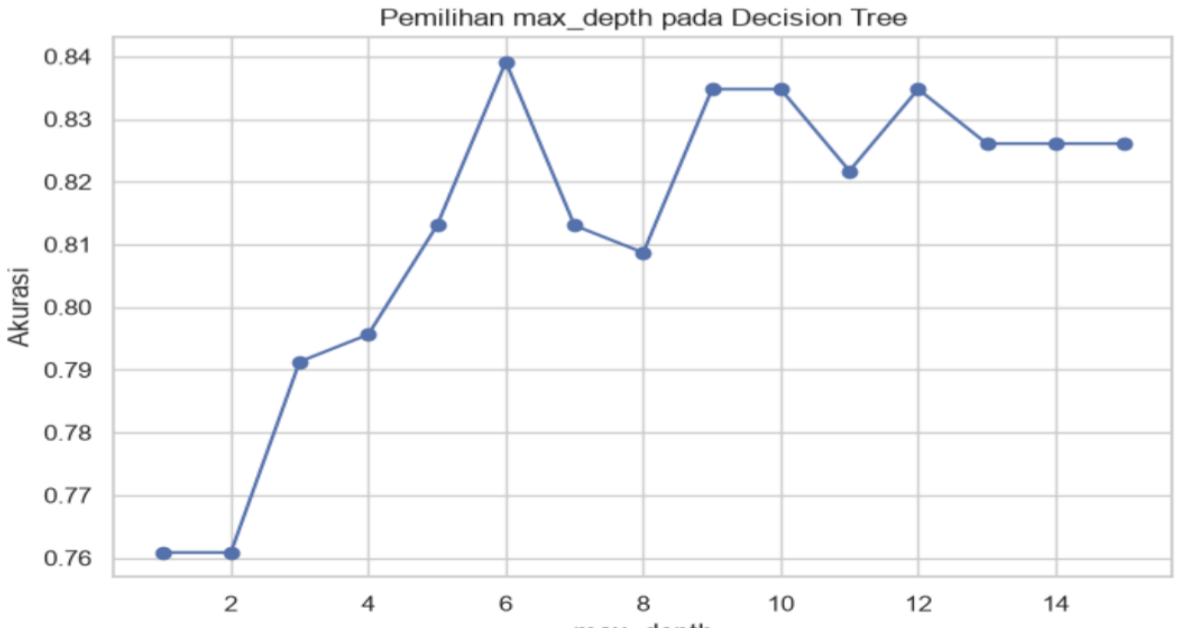
```
[15]: # 202531081_M. Fasha Bintang Pratama
depths = range(1, 16)
akurasi_dt = []

for d in depths:
    model = DecisionTreeClassifier(criterion="entropy", max_depth=d, random_state=0)
    model.fit(x_train, y_train)
    akurasi_dt.append(accuracy_score(y_test, model.predict(x_test)))

plt.figure(figsize=(8, 5))
plt.plot(depths, akurasi_dt, marker="o")
plt.xlabel("max_depth")
plt.ylabel("Akurasi")
plt.title("Pemilihan max_depth pada Decision Tree")

best_depth = depths[akurasi_dt.index(max(akurasi_dt))]
print("max_depth terbaik:", best_depth, "| akurasi:", max(akurasi_dt))
```

```
max_depth terbaik: 6 | akurasi: 0.8391304347826087
```




- depths = range(1, 16) digunakan untuk menentukan variasi kedalaman pohon keputusan yang akan diuji, mulai dari nilai 1 hingga 15.
- akurasi_dt = [] digunakan untuk membuat list kosong yang berfungsi menyimpan hasil akurasi dari setiap model Decision Tree yang diuji.
- for d in depths: digunakan untuk melakukan perulangan pada setiap nilai kedalaman pohon yang telah ditentukan, sehingga model dapat diuji dengan berbagai nilai max_depth.
- model = DecisionTreeClassifier(criterion="entropy", max_depth=d, random_state=0) digunakan untuk membuat model Decision Tree Classifier. Parameter criterion="entropy" menentukan metode pengukuran ketidakmurnian data

berdasarkan entropy, max_depth=d membatasi kedalaman maksimum pohon sesuai nilai yang sedang diuji, sedangkan random_state=0 menjaga konsistensi hasil proses yang melibatkan unsur acak.

- model.fit(x_train, y_train) digunakan untuk melatih model menggunakan data latih yang terdiri dari fitur x_train dan target y_train.
- akurasi_dt.append(accuracy_score(y_test, model.predict(x_test))) digunakan untuk menghitung akurasi model menggunakan data uji. Hasil prediksi dibandingkan dengan label sebenarnya, kemudian nilai akurasinya disimpan ke dalam list akurasi_dt.
- plt.figure(figsize=(8, 5)) digunakan untuk membuat area grafik dengan ukuran lebar 8 inci dan tinggi 5 inci.
- plt.plot(depths, akurasi_dt, marker="o") digunakan untuk membuat grafik garis yang menunjukkan perubahan nilai akurasi berdasarkan variasi kedalaman pohon. Parameter marker="o" digunakan untuk menampilkan titik pada setiap nilai kedalaman yang diuji.
- plt.xlabel("max_depth") digunakan untuk memberikan label pada sumbu horizontal (X), yaitu nilai kedalaman maksimum pohon.
- plt.ylabel("Akurasi") digunakan untuk memberikan label pada sumbu vertikal (Y), yaitu nilai akurasi model.
- plt.title("Pemilihan max_depth pada Decision Tree") digunakan untuk memberikan judul pada grafik yang menunjukkan proses pemilihan kedalaman terbaik pada algoritma Decision Tree.
- best_depth = depths[akurasi_dt.index(max(akurasi_dt))] digunakan untuk mencari nilai kedalaman pohon yang menghasilkan akurasi tertinggi. Fungsi max(akurasi_dt) mengambil nilai akurasi terbesar, sedangkan index()mencari posisi nilai tersebut dalam list. Posisi tersebut kemudian digunakan untuk mengambil nilai kedalaman yang sesuai dari depths.
- print("max_depth terbaik:", best_depth, "| akurasi:", max(akurasi_dt)) digunakan untuk menampilkan nilai max_depth terbaik beserta tingkat akurasi tertinggi yang diperoleh dari pengujian.
- Grafik menampilkan hasil evaluasi pemilihan parameter kedalaman maksimum (max_depth) pada model Decision Tree untuk menentukan tingkat kedalaman yang menghasilkan akurasi terbaik. Berdasarkan hasil pengujian, nilai akurasi mengalami peningkatan seiring bertambahnya kedalaman pohon hingga mencapai nilai tertinggi pada max_depth = 6, dengan akurasi data uji sebesar 0,8391 atau 83,91%. Setelah melewati kedalaman tersebut, nilai akurasi cenderung mengalami penurunan dan fluktuasi. Hal ini menunjukkan bahwa penambahan kedalaman pohon tidak selalu meningkatkan performa model dan berpotensi menyebabkan overfitting. Dengan demikian, max_depth = 6 dipilih sebagai parameter terbaik berdasarkan hasil pengujian karena menghasilkan akurasi tertinggi pada data uji yang digunakan.

## 3.16 Decision Tree - model akhir dan akurasi

Decision Tree - model akhir dan akurasi

[16]:
```
# 202531081_M. Fasha Bintang Pratama
from sklearn.metrics import classification_report

dt = DecisionTreeClassifier(criterion="entropy", max_depth=best_depth, random_state=0)
dt.fit(x_train, y_train)
pred_dt = dt.predict(x_test)

akurasi_dt_final = accuracy_score(y_test, pred_dt)
print("Kedalaman pohon:", dt.get_depth())
print("Akurasi Decision Tree:", akurasi_dt_final)
print(classification_report(y_test, pred_dt))
```

```
Kedalaman pohon: 6
Akurasi Decision Tree: 0.8391304347826087
              precision    recall  f1-score   support

           0       0.80      0.79      0.80        92
           1       0.86      0.87      0.87       138

    accuracy                           0.84       230
   macro avg       0.83      0.83      0.83       230
weighted avg       0.84      0.84      0.84       230
```

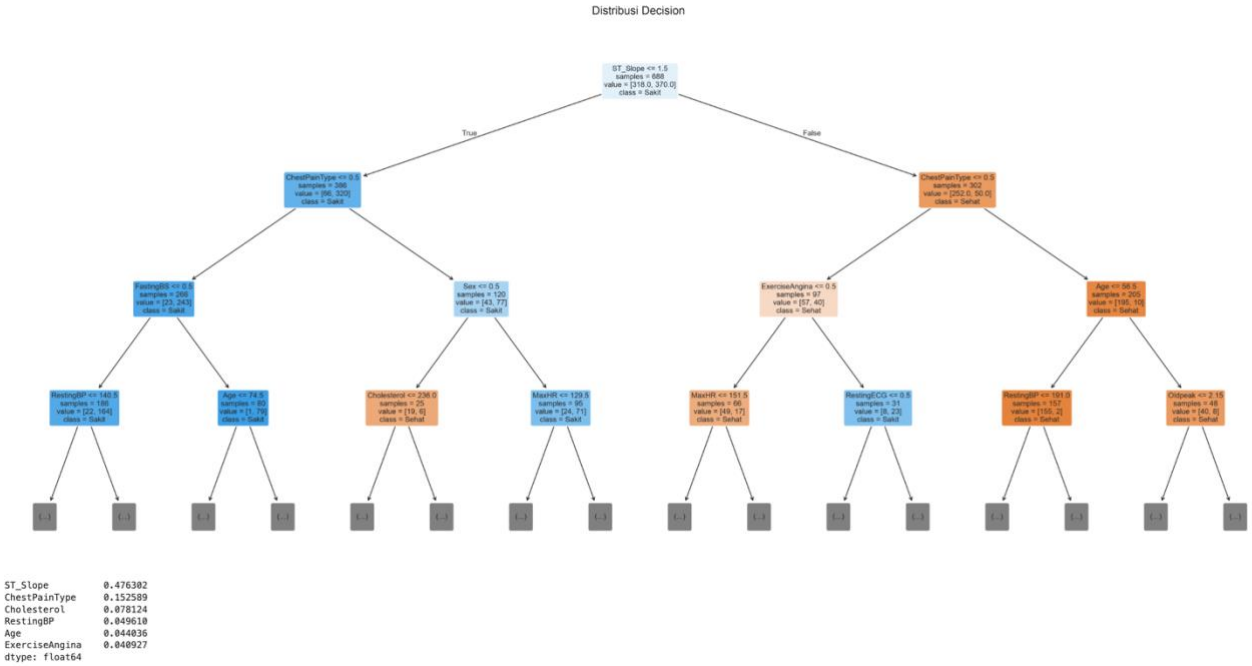
- from sklearn.metrics import classification_report digunakan untuk mengimpor fungsi classification_report dari library Scikit-learn. Fungsi ini digunakan untuk menampilkan hasil evaluasi model klasifikasi berdasarkan beberapa metrik, yaitu precision, recall, f1-score, dan support.
- dt=DecisionTreeClassifier(criterion="entropy",max_depth=best_depth,random_state=0) digunakan untuk membuat model Decision Tree Classifier dengan konfigurasi parameter yang telah ditentukan. Parameter criterion="entropy" digunakan sebagai metode pengukuran ketidakmurnian data, max_depth=best_depthmenggunakan nilai kedalaman terbaik yang telah diperoleh dari proses pengujian sebelumnya, sedangkan random_state=0 digunakan untuk menjaga konsistensi hasil.
- dt.fit(x_train, y_train) digunakan untuk melatih model Decision Tree menggunakan data latih. Data x_train berisi fitur atau atribut pasien, sedangkan y_train berisi label status penyakit jantung.
- pred_dt = dt.predict(x_test) digunakan untuk melakukan prediksi terhadap data uji menggunakan model yang telah dilatih. Hasil prediksi tersebut kemudian disimpan ke dalam variabel pred_dt.
- akurasi_dt_final = accuracy_score(y_test, pred_dt) digunakan untuk menghitung tingkat akurasi akhir model dengan membandingkan hasil prediksi pada pred_dt dengan label sebenarnya pada y_test.
- print("Kedalaman pohon:", dt.get_depth()) digunakan untuk menampilkan kedalaman pohon keputusan yang terbentuk setelah proses pelatihan model selesai.
- print("Akurasi Decision Tree:", akurasi_dt_final) digunakan untuk menampilkan nilai akurasi akhir model Decision Tree dalam memprediksi data uji.
- print(classification_report(y_test, pred_dt)) digunakan untuk menampilkan laporan evaluasi klasifikasi yang berisi nilai precision, recall, f1-score, dan support untuk setiap kelas. Laporan ini membantu mengetahui kemampuan model dalam mengklasifikasikan pasien ang memiliki maupun tidak memiliki penyakit jantung.

- Hasil keluaran classification report menunjukkan evaluasi performa model Decision Tree setelah menggunakan kedalaman pohon optimal, yaitu max_depth = 6, dengan akurasi keseluruhan sebesar 0,84 atau 84% pada 230 data uji. Secara rinci, pada kelas 0 (pasien tanpa penyakit jantung) dengan jumlah 92 sampel, model memperoleh nilai precision sebesar 0,80, recall sebesar 0,79, dan f1-score sebesar 0,80. Sementara itu, pada kelas 1 (pasien dengan penyakit jantung) dengan jumlah 138 sampel, model memperoleh precision sebesar 0,86, recall sebesar 0,87, dan f1-score sebesar 0,87. Nilai macro average dan weighted average yang berada pada kisaran 0,83 hingga 0,84 menunjukkan bahwa model memiliki performa klasifikasi yang cukup baik pada kedua kelas. Berdasarkan hasil tersebut, model Decision Tree memiliki kemampuan yang sedikit lebih tinggi dalam mengenali pasien dengan penyakit jantung dibandingkan pasien tanpa penyakit jantung. Meskipun demikian, hasil ini masih perlu dievaluasi lebih lanjut sebelum model digunakan untuk kebutuhan diagnosis medis.

## 3.17 Decision Tree - gambar pohon dan fitur penting

[17]:
```
# 202531081_M. Fasha Bintang Pratama
from sklearn import tree
plt.figure(figsize=(22, 10), dpi=300)
tree.plot_tree(
    dt,
    feature_names=x.columns,
    class_names=["Sehat", "Sakit"],
    filled=True,
    fontsize=9,
    max_depth=3,
    impurity=False,
    rounded=True,
    precision=2
)

plt.title("Distribusi Decision ", fontsize=14, pad=15)
plt.tight_layout()
```




```
ST_Slope          0.476302
ChestPainType     0.152589
Cholesterol       0.078124
RestingBP         0.049610
Age               0.044036
ExerciseAngina    0.040927
dtype: float64
```

- from sklearn import tree digunakan untuk mengimpor modul tree dari Scikit-Learn yang berfungsi untuk membuat dan menampilkan visualisasi model Decision Tree.
- plt.figure(figsize=(22, 10), dpi=300) digunakan untuk mengatur ukuran gambar menjadi 22 × 10 inci dengan resolusi 300 DPI agar visualisasi pohon keputusan terlihat lebih jelas.
- tree.plot_tree(dt, ...) digunakan untuk menampilkan struktur pohon keputusan berdasarkan model Decision Tree yang telah dilatih sebelumnya.
- feature_names=x.columns digunakan untuk menampilkan nama-nama fitur atau variabel yang digunakan dalam proses pembentukan model.
- class_names=["Sehat", "Sakit"] digunakan untuk memberikan nama pada dua kelas target, yaitu Sehat dan Sakit.
- filled=True digunakan untuk memberikan warna pada setiap kotak dalam pohon keputusan agar perbedaan kelas lebih mudah dikenali.
- fontsize=9 digunakan untuk mengatur ukuran tulisan pada setiap kotak pohon keputusan menjadi 9.
- max_depth=3 digunakan untuk membatasi tampilan visualisasi hingga kedalaman tiga tingkat agar struktur pohon lebih sederhana dan mudah dibaca. Pembatasan ini hanya memengaruhi tampilan, bukan mengubah kedalaman model yang sudah dilatih.
- impurity=False digunakan untuk menyembunyikan informasi nilai impurity atau tingkat ketidakmurnian pada setiap node dalam visualisasi.
- rounded=True digunakan untuk membuat sudut kotak pada pohon keputusan menjadi melengkung sehingga tampilannya lebih rapi.
- precision=2 digunakan untuk membatasi angka desimal yang ditampilkan pada visualisasi menjadi dua angka di belakang koma.
- plt.title("Distribusi Decision ", fontsize=14, pad=15) digunakan untuk memberikan judul pada grafik dengan ukuran tulisan 14 dan jarak judul dari grafik sebesar 15.
- plt.tight_layout() digunakan untuk mengatur tata letak visualisasi agar seluruh bagian grafik tersusun dengan rapi dan tidak saling bertumpuk.
- plt.show() digunakan untuk menampilkan hasil visualisasi pohon keputusan.
- Penting=pd.Series(dt.feature_importances_,index=x.columns).sort_values(ascending=False)digunakan untuk menghitung dan mengurutkan tingkat kepentingan setiap fitur berdasarkan model Decision Tree dari nilai tertinggi hingga terendah.
- print(penting.head(6)) digunakan untuk menampilkan enam fitur dengan nilai kepentingan tertinggi yang paling berpengaruh dalam proses pengambilan keputusan model.
- Gambar tersebut menyajikan struktur diagram Decision Tree beserta tingkat kepentingan fitur (feature importance) yang digunakan dalam mengklasifikasikan kondisi pasien. Pohon keputusan diawali dengan node akar (root node) yang menggunakan variabel ST_Slope dengan batas pemisah ≤ 1,5. Hal ini menunjukkan bahwa ST_Slope menjadi variabel utama yang digunakan model dalam membedakan pasien sehat dan pasien dengan penyakit jantung. Pada tingkatan berikutnya, proses pemisahan data melibatkan beberapa variabel lain, seperti ChestPainType, Sex, FastingBS, dan ExerciseAngina hingga mencapai node daun (leaf nodes).

  Berdasarkan hasil feature importance, ST_Slope memiliki nilai kepentingan tertinggi sebesar 0,476, diikuti oleh ChestPainType sebesar 0,153, Cholesterol sebesar 0,078, RestingBP sebesar 0,049, Age sebesar 0,044, dan ExerciseAngina sebesar 0,044. Hasil tersebut menunjukkan bahwa ST_Slope dan ChestPainType merupakan dua fitur yang memberikan kontribusi terbesar terhadap proses pengambilan keputusan model. Dengan demikian, model Decision Tree lebih banyak memanfaatkan informasi mengenai kemiringan segmen ST dan jenis nyeri dada dalam melakukan klasifikasi kondisi pasien. Namun, nilai feature importance menunjukkan kontribusi fitur terhadap keputusan model dan tidak secara langsung membuktikan hubungan sebab-akibat dengan penyakit jantung.

## 3.18 Naive Bayes - membuat model dan prediksi

### Naive Bayes - membuat model dan prediksi

[18]:
```
# 202531081_M. Fasha Bintang Pratama
from sklearn.naive_bayes import GaussianNB

nb = GaussianNB()
nb.fit(x_train, y_train)

y_pred_train = nb.predict(x_train)
y_pred_test = nb.predict(x_test)

print("Akurasi data latih:", accuracy_score(y_train, y_pred_train))
print("Akurasi data uji  :", accuracy_score(y_test, y_pred_test))
```

```
Akurasi data latih: 0.8517441860465116
Akurasi data uji  : 0.8260869565217391
```

- from sklearn.naive_bayes import GaussianNB digunakan untuk mengimpor algoritma Gaussian Naive Bayes dari library Scikit-Learn yang berfungsi untuk melakukan klasifikasi berdasarkan perhitungan probabilitas.
- nb = GaussianNB() digunakan untuk membuat objek model Gaussian Naive Bayes yang nantinya akan digunakan dalam proses klasifikasi data pasien.
- nb.fit(x_train, y_train) digunakan untuk melatih model menggunakan data latih. Model mempelajari pola dari fitur-fitur data pasien dan kategori target, yaitu Sehat atau Sakit.
- y_pred_train = nb.predict(x_train) digunakan untuk melakukan prediksi terhadap data latih sehingga dapat diketahui hasil klasifikasi model pada data yang digunakan selama proses pelatihan.
- y_pred_test = nb.predict(x_test) digunakan untuk melakukan prediksi terhadap data uji yang belum digunakan dalam proses pelatihan model.
- print("Akurasidatalatih:",accuracy_score(y_train,y_pred_train)) digunakan untuk menghitung dan menampilkan tingkat akurasi model pada data latih dengan membandingkan hasil prediksi terhadap label sebenarnya.
- print("Akurasidata uji :", accuracy_score(y_test, y_pred_test)) digunakan untuk menghitung dan menampilkan tingkat akurasi model pada data uji. Hasil ini digunakan untuk mengetahui kemampuan model dalam mengklasifikasikandata baru.
- Hasil menunjukkan performa evaluasi model Gaussian Naive Bayes dalam melakukan klasifikasi penyakit jantung. Model memperoleh akurasi data latih sebesar 0,8517 atau 85,17% dan akurasi data uji sebesar 0,8261 atau 82,61%. Selisih sekitar 2,56% antara kedua nilai akurasi tersebut menunjukkan bahwa performa model pada data latih dan data uji relatif berdekatan. Hal ini mengindikasikan bahwa model memiliki kemampuan generalisasi yang cukup baik dan tidak menunjukkan perbedaan performa yang terlalu besar antara kedua kelompok data. Dengan demikian, model Gaussian Naive Bayes mampu melakukan klasifikasi dengan hasil yang cukup konsisten pada data yang digunakan dalam pengujian.

## 3.19 Naive Bayes - evaluasi dengan confusion matrix

### Naive Bayes - evaluasi dengan confusion matrix

[19]:
```
# 202531081_M. Fasha Bintang Pratama
from sklearn.metrics import confusion_matrix

cm_train = confusion_matrix(y_train, y_pred_train)
cm_test = confusion_matrix(y_test, y_pred_test)

print("evaluasi data uji:")
print(cm_test)
print(classification_report(y_test, y_pred_test, target_names=["Sehat", "Sakit"]))

plt.figure(figsize=(11, 4))

plt.subplot(1, 2, 1)
sns.heatmap(cm_train, annot=True, fmt="d", cmap="Blues", xticklabels=["Sehat", "Sakit"], yticklabels=["Sehat", "Sakit"])
plt.title("Confusion Matrix - Training")
plt.xlabel("Predict Label")
plt.ylabel("True Label")

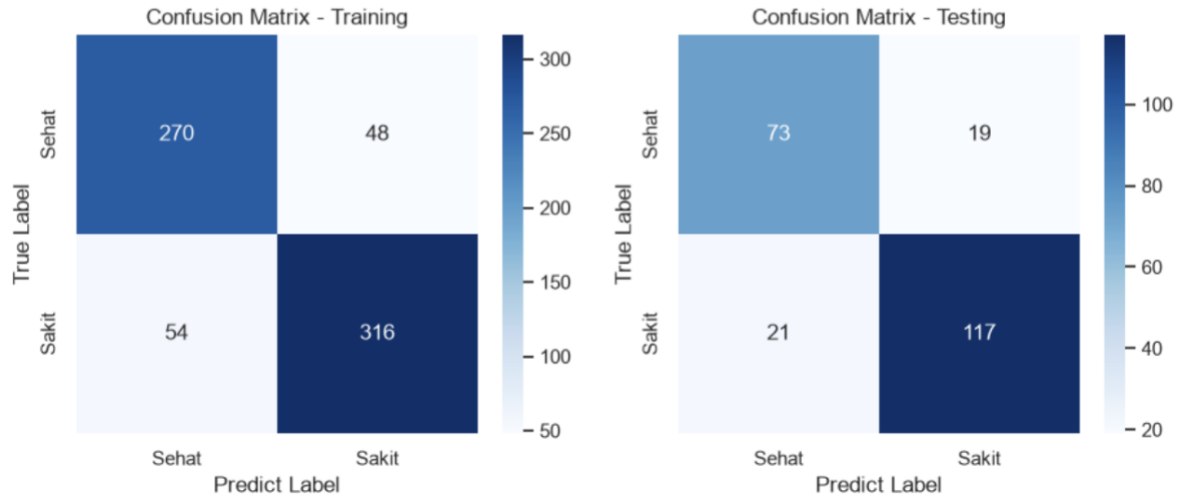
plt.subplot(1, 2, 2)
sns.heatmap(cm_test, annot=True, fmt="d", cmap="Blues", xticklabels=["Sehat", "Sakit"], yticklabels=["Sehat", "Sakit"])
plt.title("Confusion Matrix - Testing")
plt.xlabel("Predict Label")
plt.ylabel("True Label")
```

```
evaluasi data uji:
[[ 73  19]
 [ 21 117]]
              precision    recall  f1-score   support

       Sehat       0.78      0.79      0.78        92
       Sakit       0.86      0.85      0.85       138

    accuracy                           0.83       230
   macro avg       0.82      0.82      0.82       230
weighted avg       0.83      0.83      0.83       230
```

[19]:
```
Text(569.4121636284721, 0.5, 'True Label')
```




- from sklearn.metrics import confusion_matrix digunakan untuk mengimpor fungsi confusion matrix yang berfungsi mengevaluasi hasil klasifikasi model berdasarkan perbandingan antara label sebenarnya dan hasil prediksi.
- cm_train = confusion_matrix(y_train, y_pred_train) dan cm_test = confusion_matrix(y_test, y_pred_test) digunakan untuk menghitung confusion matrix pada data latih dan data uji. Hasilnya menunjukkan jumlah prediksi yang benar maupun salah pada setiap kelas.

- print("evaluasi data uji:"), print(cm_test), dan print(classification_report(...)) digunakan untuk menampilkan hasil evaluasi data uji, termasuk nilai precision, recall, f1-score, dan jumlah data pada masing-masing kelas Sehat dan Sakit.
- plt.figure(figsize=(11, 4)) digunakan untuk mengatur ukuran gambar yang akan menampilkan dua visualisasi confusion matrix.
- plt.subplot(1, 2, 1) dan plt.subplot(1, 2, 2) digunakan untuk membagi tampilan menjadi dua grafik, yaitu confusion matrix data latih dan data uji.
- sns.heatmap(...) digunakan untuk menampilkan confusion matrix dalam bentuk heatmap berwarna. Angka di dalam kotak menunjukkan jumlah hasil prediksi, sedangkan label Sehat dan Sakit digunakan untuk memperjelas kategori klasifikasi.
- plt.title(), plt.xlabel(), dan plt.ylabel() digunakan untuk memberikan judul serta keterangan pada setiap grafik. Sumbu horizontal menunjukkan label hasil prediksi, sedangkan sumbu vertikal menunjukkan label sebenarnya.
- Hasil keluaran tersebut menyajikan evaluasi performa model Gaussian Naive Bayes melalui Confusion Matrix pada data latih dan data uji serta classification report terhadap 230 data uji. Berdasarkan Confusion Matrix - Testing, model berhasil mengklasifikasikan 73 sampel Sehat dan 117 sampel Sakit dengan benar. Sementara itu, terdapat 19 sampel Sehat yang diprediksi sebagai Sakit dan 21 sampel Sakit yang diprediksi sebagai Sehat. Hasil classification report menunjukkan bahwa kelas Sehat memperoleh nilai precision sebesar 0,78 dan recall sebesar 0,79, sedangkan kelas Sakit memperoleh precision sebesar 0,86 dan recall sebesar 0,85. Secara keseluruhan, model menghasilkan akurasi sebesar 0,83 atau 83%, dengan nilai macro average dan weighted average berada pada kisaran 0,82–0,83. Hasil ini menunjukkan bahwa model memiliki kemampuan klasifikasi yang cukup baik, terutama dalam mengenali pasien dengan

## 3.20 Decision Tree - confusion matrix

Agar bisa dibandingkan dengan Naive Bayes, confusion matrix juga dibuat untuk Decision Tree pada data uji.

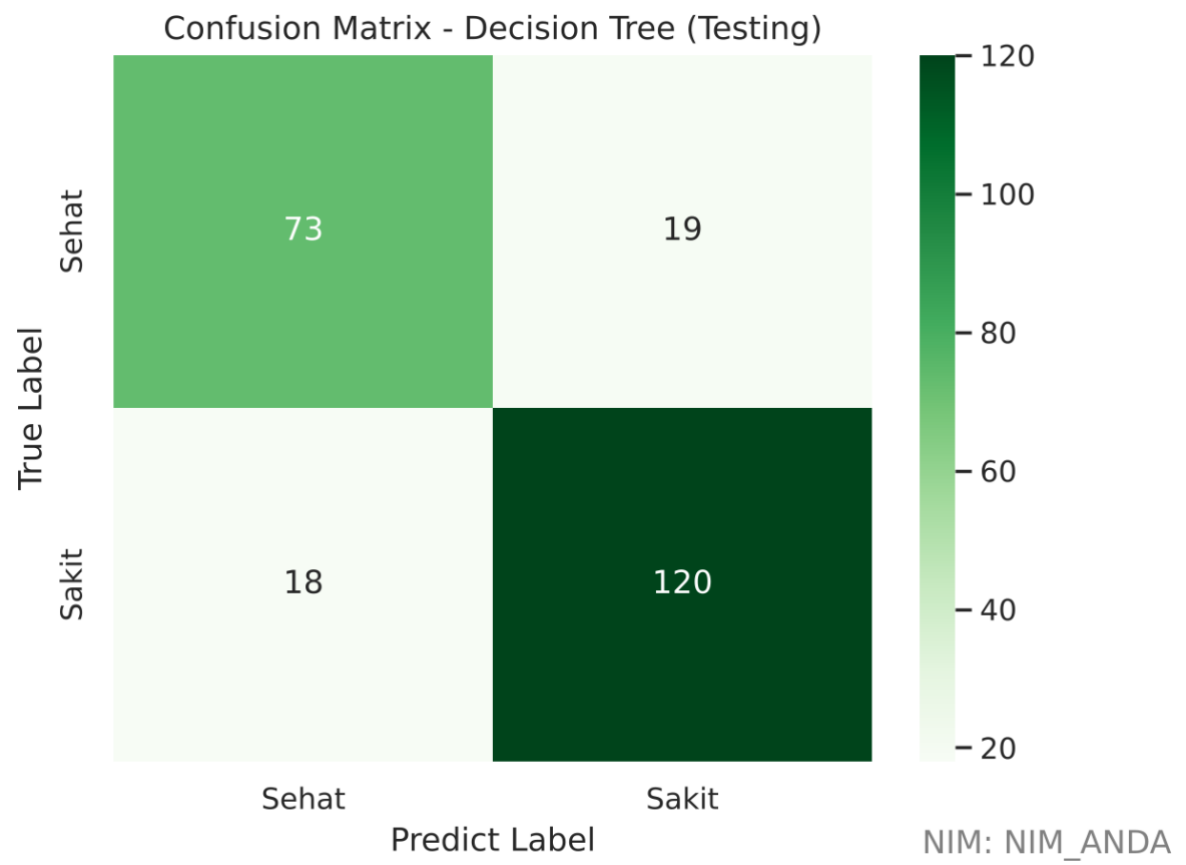
***Kode:***

```
print("NIM :", nim)
cm_dt = confusion_matrix(y_test, pred_dt)
print(cm_dt)


sns.heatmap(cm_dt, annot=True, fmt="d", cmap="Greens", xticklabels=["Sehat", "Sakit"], yticklabels=["Sehat",
"Sakit"])
plt.title("Confusion Matrix - Decision Tree (Testing)")
plt.xlabel("Predict Label")
plt.ylabel("True Label")
plt.figtext(0.99, 0.01, "NIM: " + nim, ha="right", color="gray")
plt.show()
```

***Output:***

```
NIM : NIM_ANDA
[[ 73  19]
 [ 18 120]]
```




*Gambar/Output 3.20. Decision Tree - confusion matrix*

## 3.21 Mencoba memprediksi satu pasien

```
[21]: # 202531081_M. Fasha Bintang Pratama
      pasien = x_test.iloc[[0]]
      print(pasien)
      print("Kondisi sebenarnya    :", y_test.iloc[0])
      print("Prediksi Decision Tree:", dt.predict(pasien)[0])
      print("Prediksi Naive Bayes  :", nb.predict(pasien)[0])
```

```
     Age  Sex  ChestPainType  RestingBP  Cholesterol  FastingBS  RestingECG  \
306   55    1              0        115          237          1           1

     MaxHR  ExerciseAngina  Oldpeak  ST_Slope
306    155               0      0.1         1
Kondisi sebenarnya    : 1
Prediksi Decision Tree: 1
Prediksi Naive Bayes  : 1
```

- pasien = x_test.iloc[[0]] digunakan untuk mengambil satu data pasien pertama dari data uji sebagai sampel yang akan diuji menggunakan kedua model klasifikasi.
- print(pasien) digunakan untuk menampilkan informasi fitur atau variabel dari pasien yang dipilih.
- print("Kondisi sebenarnya :", y_test.iloc[0]) digunakan untuk menampilkan kondisi sebenarnya dari pasien berdasarkan label pada data uji.
- print("Prediksi Decision Tree:", dt.predict(pasien)[0]) digunakan untuk menampilkan hasil prediksi kondisi pasien menggunakan model Decision Tree.

- print("Prediksi Naive Bayes :", nb.predict(pasien)[0]) digunakan untuk menampilkan hasil prediksi kondisi pasien menggunakan model Gaussian Naive Bayes.
- Gambar ini menyajikan hasil pengujian inferensi pada sampel data individu (pasien indeks ke-306) untuk membandingkan hasil prediksi model *Decision Tree* dan *Naive Bayes* terhadap kondisi aktual (*ground truth*). Sampel pasien tersebut memiliki profil medis antara lain usia 55 tahun (Age), jenis kelamin laki-laki(Sex:1),tipe nyeri dada 0(ChestPainType), RestingBP sebesar 115, Cholesterol sebesar 237, MaxHR sebesa r 155, serta ST_Slope bernilai 1. Hasil pengujian menunjukkan bahwa nilai Kondisi sebenarnya adalah 1 (positif terdiagnosa penyakit jantung). Kedua algoritma yang diuji, baik Prediksi Decision Tree: 1 maupun Prediksi Naive Bayes: 1, berhasil memprediksi label kelas tersebut secara tepat (*True Positive*), yang mengonfirmasi bahwa kedua model mampu mengenali kombinasi pola fitur medis pasien dengan akurat pada tingkat individu.

## 3.22 Perbandingan Decision Tree dan Naive Bayes

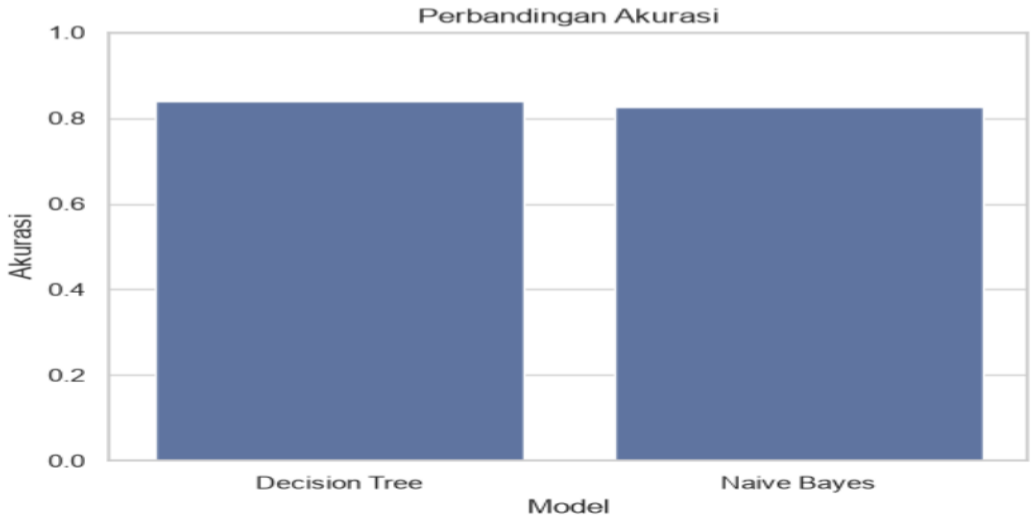
**Perbandingan Decision Tree dan Naive Bayes**

```
[22]: # 202531081_M. Fasha Bintang Pratama
akurasi_nb_final = accuracy_score(y_test, y_pred_test)

hasil = pd.DataFrame({
    "Model": ["Decision Tree", "Naive Bayes"],
    "Akurasi": [akurasi_dt_final, akurasi_nb_final]
})
print(hasil)

sns.barplot(x="Model", y="Akurasi", data=hasil)
plt.ylim(0, 1)
plt.title("Perbandingan Akurasi")
```

```
           Model   Akurasi
0  Decision Tree  0.839130
1    Naive Bayes  0.826087
[22]: Text(0.5, 1.0, 'Perbandingan Akurasi')
```




- akurasi_nb_final = accuracy_score(y_test, y_pred_test) digunakan untuk menghitung akurasi akhir model Gaussian Naive Bayes berdasarkan perbandingan antara label sebenarnya dan hasil prediksi pada data uji.
- hasil = pd.DataFrame({...}) digunakan untuk membuat tabel perbandingan yang berisi nama dua model klasifikasi, yaitu Decision Tree dan Naive Bayes, beserta nilai akurasi masing-masing model.

- print(hasil) digunakan untuk menampilkan tabel perbandingan akurasi kedua model.
- sns.barplot(x="Model", y="Akurasi", data=hasil) digunakan untuk membuat diagram batang yang memperlihatkan perbandingan tingkat akurasi antara model Decision Tree dan Gaussian Naive Bayes.
- plt.ylim(0, 1) digunakan untuk mengatur batas sumbu vertikal grafik dari 0 hingga 1 atau setara dengan 0% sampai 100%.
- plt.title("Perbandingan Akurasi") digunakan untuk memberikan judul pada diagram batang agar tujuan visualisasi lebih mudah dipahami.
- Tabel dan grafik batang di atas menyajikan perbandingan tingkat akurasi antara algoritma Decision Tree dan Gaussian Naive Bayes dalam mengklasifikasikan penyakit jantung berdasarkan data uji. Berdasarkan hasil evaluasi, model Decision Tree memperoleh akurasi sebesar 0,8391 atau 83,91%, sedangkan model Gaussian Naive Bayes memperoleh akurasi sebesar 0,8261 atau 82,61%. Terdapat selisih akurasi sekitar 1,3% yang menunjukkan bahwa performa kedua model relatif berdekatan. Meskipun demikian, Decision Tree memiliki hasil akurasi sedikit lebih tinggi dibandingkan Gaussian Naive Bayes. Hal ini menunjukkan bahwa Decision Tree memberikan hasil klasifikasi yang sedikit lebih baik pada data uji yang digunakan. Namun, penilaian model terbaik tidak hanya didasarkan pada akurasi, tetapi juga perlu mempertimbangkan precision, recall, dan f1-score.

# BAB IV

# PENUTUP

## 4.1 Kesimpulan

Berdasarkan landasan teori dan hasil praktikum, dapat disimpulkan bahwa:

1. Dataset Heart Disease yang digunakan terdiri dari 918 data dan 12 kolom. Berdasarkan pemeriksaan awal, tidak ditemukan data kosong maupun data duplikat. Namun, terdapat nilai 0 pada kolom Cholesterol sebanyak 172 data dan RestingBP sebanyak 1 data yang kemudian ditangani menggunakan nilai median agar data dapat digunakan dalam proses pemodelan.
2. Dari hasil EDA, diketahui bahwa jumlah pasien dengan kategori penyakit jantung (kelas 1) sebanyak 508 data atau sekitar 55,3%, sedangkan pasien tanpa penyakit jantung (kelas 0) sebanyak 410 data atau sekitar 44,7%. Beberapa fitur yang memiliki hubungan cukup kuat dengan HeartDisease adalah ST_Slope, ExerciseAngina, Oldpeak, MaxHR, dan ChestPainType.
3. Model Decision Tree bawaan menghasilkan akurasi yang kemudian ditingkatkan melalui pengaturan parameter max_depth. Setelah dilakukan pengujian terhadap beberapa kedalaman pohon, nilai max_depth terbaik yang diperoleh adalah 6 dengan akurasi data uji sebesar 83,91%. Fitur yang memiliki tingkat kepentingan tertinggi dalam model adalah ST_Slope, ChestPainType, dan Cholesterol.
4. Algoritma Gaussian Naive Bayes digunakan sebagai metode pembanding. Model ini memperoleh akurasi data latih sebesar 85,17% dan akurasi data uji sebesar 82,61%. Evaluasi menggunakan confusion matrix menunjukkan bahwa model mampu mengklasifikasikan sebagian besar data pasien dengan benar, baik pada kategori sehat maupun sakit.
5. Decision Tree memperoleh akurasi sekitar 1,30 poin persentase lebih tinggi dibandingkan Gaussian Naive Bayes. Dengan demikian, Decision Tree memberikan hasil yang sedikit lebih baik pada dataset yang digunakan dalam praktikum ini. Namun, kedua algoritma memiliki performa yang relatif berdekatan dalam melakukan klasifikasi penyakit jantung.

Saran untuk pengembangan berikutnya, pengujian model dapat dilakukan menggunakan metode cross validation agar hasil evaluasi lebih konsisten dan tidak hanya bergantung pada satu kali pembagian data. Selain itu, dapat dilakukan pengujian parameter yang lebih beragam serta membandingkan algoritma lain untuk mengetahui metode yang memberikan hasil klasifikasi lebih baik. Penggunaan fitur dan data yang lebih lengkap juga dapat dipertimbangkan agar hasil prediksi semakin optimal.

# DAFTAR PUSTAKA

Azizah, S. R., Herteno, R., Farmadi, A., Kartini, D., & Budiman, I. (2023). Kombinasi seleksi fitur berbasis filter dan wrapper menggunakan Naive Bayes pada klasifikasi penyakit jantung. *Jurnal Teknologi Informasi dan Ilmu Komputer, 10*(6). https://doi.org/10.25126/jtiik.1067467

Bahri, C. A., Zafira, Z. T., Ayuningtiyas, P., Al-Farisy, M. H., Alfarizi, M., Tania, K. D., & Meiriza, A. (2025). Analisis faktor risiko pemicu serangan jantung di Indonesia, menggunakan metode klasifikasi (Decision Tree, Naive Bayes, dan Random Forest). *JATI (Jurnal Mahasiswa Teknik Informatika, 9*(4). https://doi.org/10.36040/jati.v9i4.13945

Hirwono, B., Hermawan, A., & Avianto, D. (2023). Implementasi metode Naïve Bayes untuk klasifikasi penderita penyakit jantung. *Jurnal JTIK (Jurnal Teknologi Informasi dan Komunikasi), 7*(3), 450–457. https://doi.org/10.35870/jtik.v7i3.910

Islam, M. A., et al. (2024). Precision healthcare: A deep dive into machine learning algorithms and feature selection strategies for accurate heart disease prediction. *Computers in Biology and Medicine, 176*, 108432. https://doi.org/10.1016/j.compbiomed.2024.108432

Kumar, R., Garg, S., Kaur, R., Johar, M. G. M., Singh, S. V., Menon, S. V., Kumar, P., Hadi, A. M., Hasson, S. A., & Lozanović, J. (2025). A comprehensive review of machine learning for heart disease prediction: Challenges, trends, ethical considerations, and future directions. *Frontiers in Artificial Intelligence, 8*, 1583459. https://doi.org/10.3389/frai.2025.1583459

Wibowo, A. C., Lestari, S. A., & Nurchim. (2024). Analisis penggunaan machine learning dalam klasifikasi penentuan penyakit jantung. *Simtek: Jurnal Sistem Informasi dan Teknik Komputer, 9*(2), 97–101. https://doi.org/10.51876/simtek.v9i2.395